



Architektura dla Front-endu

Projektowanie, Praktyki, Narzędzia i Wzorce



Szkolenie

- Plan szkolenia i cele
- Aktualne obowiązki
- Pytania, dyskusje, potrzeby
- Elastyczny program szkoleniowy



Mateusz Kulesza

- Senior Software Developer
- Enterprise Architect
- Solution Architect, Project Manager
- Consultant and Trainer

Architektura: fundamenty i rola

Architektura: fundamenty i rola

- Architektura to decyzje trudne lub kosztowne do zmiany
- Diagram jest tylko reprezentacją architektury — architektura istnieje też bez dokumentacji i architekta
- Kontekst biznesowy i organizacyjny zmienia, co jest „dobrą” odpowiedzią
- Ten deck: czym jest architektura, czym NIE jest, i jaka jest rola architekta w zespole

Architektura systemu to:

- struktura systemu — kluczowe elementy i ich odpowiedzialności
- relacje i zależności między elementami
- istotne właściwości systemu
- zasady jego budowy i ewolucji

Nie chodzi wyłącznie o:

- diagram komponentów
- wybór frameworka
- podział na mikroserwisy
- „stos technologiczny”

Kluczowe pytanie: co będzie trudno lub kosztownie zmienić później?

THIS IS NOT A PIPE.



Ceci n'est pas une pipe.

Architektura $\neq$ diagram

Architektura istnieje również wtedy, gdy:

- nie ma dokumentacji,
- nie ma architekta,
- nikt nie podjął formalnych decyzji,
- system powstał „organicznie”.

Przykład — to jeszcze prawie nic nie mówi:

Frontend → Backend → Database

Potrzebujemy wiedzieć m.in.: kto może wywoływać kogo, kto jest właścicielem danych, gdzie są granice odpowiedzialności, jakie zależności są dozwolone, co można wdrażać niezależnie, gdzie wymagamy spójności.

Każdy system ma swoją architekturę

Niektóre architektury są oczywiste i widoczne, wiele innych nie bardzo.

- opisowa architektura systemu ostatecznie znajduje się w jego kodzie wykonywalnym
- przed zbudowaniem systemu warto jasno określić jego normatywną architekturę
- brak świadomego projektowania nie oznacza braku architektury

Decyzje lokalne → kolejne wyjątki → zależności przypadkowe → architektura emergentna

Typowe symptomy: każdy moduł zna każdy, wspólna baza danych jest „API” między systemami, „tego nie możemy ruszyć”, wdrożenie jednego elementu wymaga wdrożenia pięciu innych, nikt nie wie dlaczego coś działa właśnie tak.

Architektura to opis albo przepis — nie każda architektura jest dobra.

Intended vs Implemented vs Runtime

Perspektywa	Pytanie
Intended	Jak system miał działać?
Implemented	Co faktycznie napisaliśmy?
Runtime	Jak system zachowuje się na produkcji?

Przykład:

```
Zamierzone:  Order → Payment
Kod:         Order → Integration Layer → Payment
Produkcja:   Order → Integration Layer → Retry Queue → Payment
              → Timeout → Manual reconciliation
```

W praktyce te trzy obrazy rzadko są identyczne — architektura produkcyjna często jest bogatsza od diagramów projektowych.

Kod nie pokazuje całej architektury

Na architekturę wpływają również: infrastruktura, sieć, deployment, konfiguracja, dane, kolejki, cache, retry, autoscaling, uprawnienia, monitoring, procedury operacyjne.

Kod może sugerować:

```
Service A → Service B
```

Produkcja:

```
Service A → API Gateway → Service Mesh → Retry  
→ Circuit Breaker → Service B → Database Replica
```



Architektura a projekt: koszt zmiany jako kryterium

Architektura jest projektem, jednak nie każdy projekt jest architekturą — wiele decyzji projektowych pozostaje otwartych i należy do dalszych etapów prac (lub do implementacji).

Praktyczne kryterium: jak kosztowna będzie zmiana tej decyzji?

- Implementacyjna — nazwa zmiennej, lokalna funkcja, sposób iteracji po kolekcji
- Projektowa — struktura klasy, API komponentu, model formularza
- Architektoniczna — podział systemu, własność danych, sync vs async, model

„Architectural significance” zależy od kontekstu

Ta sama decyzja może być mało lub bardzo istotna architektonicznie.

Przykład: wybór biblioteki

- Mało architektoniczne — `date-fns` vs `dayjs` , jeśli używana lokalnie w jednym module
- Bardzo architektoniczne — `React` vs `Angular` , jeśli: organizacja ma 30 zespołów, framework determinuje platformę, istnieje wspólny design system, trzeba utrzymać system 10 lat

Architektoniczność nie wynika z nazwy technologii, lecz z konsekwencji decyzji.

Architektura to proces, nie stan końcowy

Antywzorzec:

Analiza → Architektura → Implementacja → Gotowe

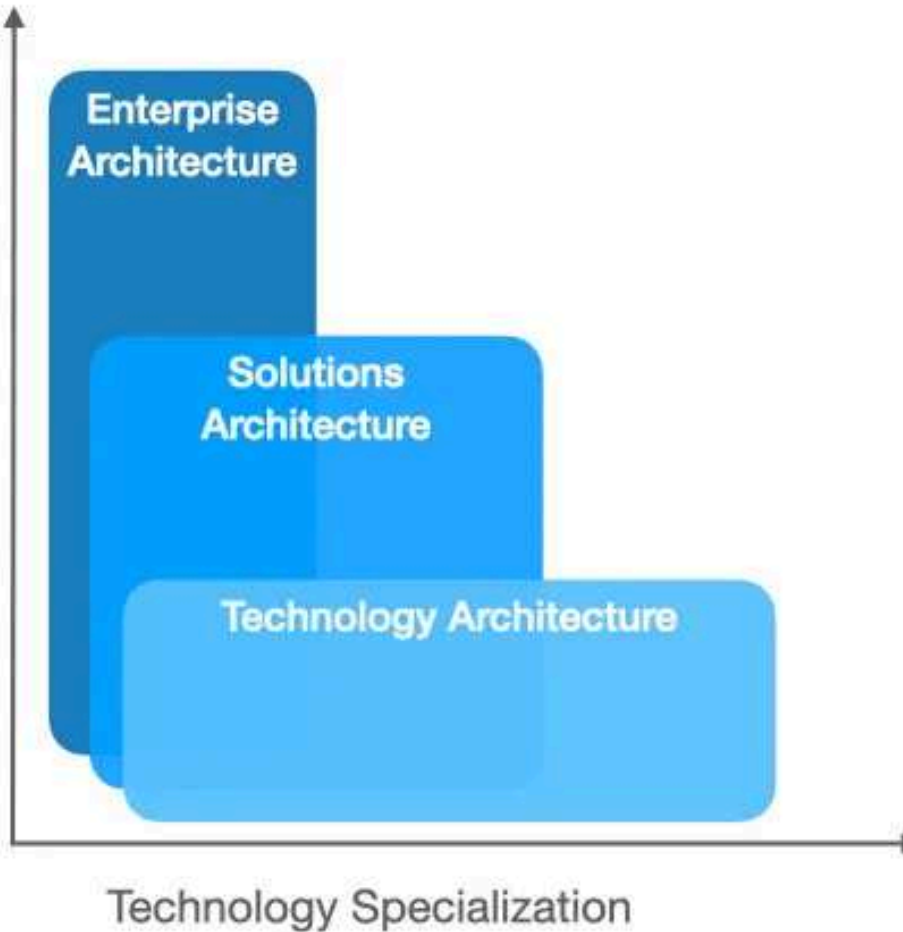
Rzeczywistość:

poznajemy problem → podejmujemy decyzję → implementujemy
→ obserwujemy → uczymy się → zmieniamy architekturę ↻

Dlaczego architektura się zmienia? Zmieniają się: wymagania, skala, użytkownicy, organizacja, zespoły, technologie, integracje, przepisy, ryzyka, koszty.

1 zespół → modular monolith → 5 zespołów
→ wybrane niezależne moduły → distributed system

Nie dlatego, że „microservices are better”, ale dlatego, że zmieniły się drivers.



Rola architekta w zespole

- architektura to odpowiedzialność realizowana przez jedną osobę lub cały zespół
- formalny architekt vs developer podejmujący decyzje architektoniczne
- architektura scentralizowana, federacyjna i zespołowa
- facylituje decyzje, nie tylko je „zatwierdza”
- łączy biznes, dev, security i operations
- buduje guardrails zamiast ręcznej policii standardów
- dba o ślad: driver → decyzja → konsekwencja

Odpowiedzialności i kompetencje architekta

Odpowiedzialności

- rozpoznawanie driverów i interesariuszy, zadawanie pytań ujawniających wymagania i ryzyka
- przygotowywanie oraz porównywanie wariantów, wyjaśnianie kompromisów i konsekwencji
- dokumentowanie kontekstu i uzasadnienia, definiowanie zasad, granic i guardrails
- weryfikowanie założeń za pomocą prototypów i spike'ów
- współpraca przy implementacji, obserwowanie produkcji i korygowanie architektury

Kompetencje

- wiedza techniczna i rozumienie ograniczeń technologii
- modelowanie domeny i procesów biznesowych
- analiza ryzyka i podejmowanie decyzji przy niepełnych danych
- komunikacja, facylitacja i rozwiązywanie konfliktów
- myślenie systemowe i rozumienie zależności organizacyjnych

Czego architekt nie powinien robić

- samodzielnie podejmować każdej decyzji technicznej
- być wyłącznie bramką akceptacyjną lub „policją standardów”
- projektować bez kontaktu z kodem, zespołem i środowiskiem produkcyjnym
- narzucać technologii na podstawie popularności albo własnego CV
- produkować dokumentacji bez określonego odbiorcy i zastosowania
- optymalizować system pod hipotetyczne problemy bez uzasadnienia w driverach

Typowe antywzorce: Ivory Tower Architect, Architecture Astronaut, architekt jako wąskie gardło, Big Design Up Front, diagramy niezgodne z implementacją, decyzje bez właściciela i uzasadnienia, „standard organizacyjny” używany zamiast analizy kontekstu.

Mini-lab (Retail Operations Portal)

1. Wskaż 3 decyzje „kosztowne do zmiany”
2. Rozróżnij: decyzja projektowa vs architektoniczna
3. Dla każdej zapisz potencjalny wpływ na Day 2/Day 3

Retail Operations Portal: kontekst case study

Retail Operations Portal: kontekst case study

- brownfieldowy monolit React 18 + TypeScript — startowy punkt całego warsztatu, przed jakąkolwiek migracją
- celowo zbudowany z antywzorcami modularności — materiał do analizy, podziału na moduły i ekstrakcji
- bez shella, bez Module Federation, bez remote'ów na wejściu — to buduje się od zera podczas warsztatu
- repo: `experiments/retail-operations-portal/` — niezależne od innych folderów w workspace

Decyzje technologiczne (case study)

Decyzja	Uzasadnienie (krótko)
React 18 + TypeScript	wymagane przez case study; TS ułatwia bezpieczne refaktory na żywo
Vite	najszybszy dev server/HMR na start — bundler pod federację dochodzi osobno, później
react-router (data mode)	routing wielostronicowy, wielu potencjalnych właścicieli tras
Tailwind CSS + shadcn/ui, domyślny motyw	komponenty jako kod; start bez customizacji tokenów — to osobny temat
Redux Toolkit — jeden globalny store	celowy antywzorzec: „store jako baza danych całego frontendu”

Zasada „minimalnie, ale wystarczająco”

Czego świadomie NIE ma w starcie:

- warstwy repository/DI, wewnętrznych „frameworków”
- i18n, testów, storybooka, CSS-in-JS
- własnego routera ani customowego state managera zamiast RTK
- prawdziwego API, serwera, autoryzacji

Każdy ekran Tier 2/3 (patrz dalej) ma tylko tyle kodu, ile potrzeba, by pokazać **jedną** konsekwencję architektoniczną — nie pełny produkt.

Struktura startowa (antywzorzec kontrolowany)

```
src/  
  main.tsx / App.tsx / routes.tsx / index.css  
  layout/      # AppLayout, Header (UserMenu, NotificationBell, GlobalSearch), Sidebar  
  components/  
    ui/        # wygenerowane przez shadcn (button, table, dialog...)  
    ...        # DataTable, StatusBadge, KpiCard, PricingRuleForm, ...  
  pages/      # 1 plik na route – cross-importuje store/components z wielu domen  
  store/  
    store.ts   # JEDEN configureStore łączący WSZYSTKIE slice'y ("god store")  
    <domena>Slice.ts  
  api/        # mock backend: delay.ts + 1 plik per domena  
  mock-data/  # *.json fixtures, 1 per domena
```

- podział wg warstwy technicznej, nie domeny; jeden globalny store; brak izolacji publicznego API modułów
- **zaszyte smelle:** `DashboardPage` importuje selektory z 3+ slice'ów; `PromotionWizardPage` sięga do `warehouseSlice` z pominięciem własnej domeny; brak `index.ts` per folder — wszystko importowalne z każdego miejsca

Mock backend — kształt

Jeden plik JSON fixture na domenę, ładowany do `Map` przy starcie modułu; odczyt/zapis z opóźnieniem symulującym sieć. Brak fetch/HTTP, brak MSW.

```
// src/api/delay.ts
export const delay = (ms = 200) => new Promise((r) => setTimeout(r, ms));

// src/api/products.ts
import raw from "../mock-data/products.json";
import { delay } from "../delay";

const db = new Map(raw.map((p) => [p.id, p]));

export async function listProducts() {
  await delay();
  return [...db.values()];
}

export async function updateProduct(id: string, patch: Partial<Product>) {
  await delay();
  const next = { ...db.get(id), ...patch };
  db.set(id, next);
  return next;
}
```

Ekran — Tier 1: rdzeń procesu promocji

Route	Ekran	Dlaczego istnieje
/	Dashboard	god-component agregujący dane z 3+ domen
/products	Lista produktów	kandydat na pierwszą ekstrakcję
/products/:id	Szczegóły produktu	odwołanie do Warehouse (stock) — cross-domain coupling
/promotions	Lista promocji	filtr po statusie (draft/pending/active/archived)
/promotions/new	Kreator promocji, 5 kroków	produkty → reguły/ceny → walidacja → akceptacja → aktywacja
/promotions/:id	Szczegóły promocji	oś czasu + slot pod przyszły widget „Performance”

Ekran — Tier 2: bufor

Route	Ekran	Zastosowanie
<code>/suppliers</code>	Dostawcy	kolejny kandydat na ekstrakcję — lub kontrprzykład, który zostaje nietknięty
<code>/orders</code>	Zamówienia	CRUD → CQRS (<code>approveOrder</code> / <code>cancelOrder</code>), niezwiązane z promocjami
<code>/warehouse</code>	Stany magazynowe	cross-domain reference z Product Details
<code>/categories</code>	Kategorie produktów	pytanie o granulację: część Products czy osobny moduł?
<code>/customer-segments</code>	Segmenty klientów	inny zespół właścicielski (Marketing)

Ekran — Tier 3: świadomie nieobecne

Dobudowywane na żywo w trakcie warsztatu:

- logowanie, guardy i prawdziwy auth context
- UI panelu Feature Flags (model istnieje od startu, UI nie)
- pełna implementacja `/reports`
- mały widget w innym frameworku (np. Vue) osadzony w `/promotions/:id`
- shell i wydzielone remote'y — cała infrastruktura federacji

Współdzielone widgets i listy

Widgets (reużywane celowo — materiał do dyskusji o couplingu):

`DataTable` (sort/filtr/paginacja/zaznaczanie/bulk toolbar), `StatusBadge`, `KpiCard`,
`PromotionStatusTimeline`, `ProductPicker`, `PricingRuleForm` (duplikacja: standalone + krok wizarda),
`ValidationResultsPanel`, `ApprovalActionBar`, `NotificationBell`, `UserMenu`, `GlobalSearch`,
`FileImportWidget`, `AuditTrailList`, `ConfirmationModal`, `AppErrorBoundary`.

Listy (instancje `DataTable`): `ProductsList`, `SuppliersList`, `OrdersList`, `WarehouseStockList`, `PromotionsList`,
`ValidationQueueList`, `ApprovalsQueueList`, `CategoriesList`, `CustomerSegmentsList`, `ReturnsList`,
`NotificationsList`, `AuditLogList`, `UsersList`.

Akcje — CRUD vs CQRS

- proste CRUD: `selectProductsForPromotion`, `addPricingRule` / `removePricingRule`,
`markNotificationRead`, `assignUserRole`
- komendy procesowe: `submitPromotionForValidation`, `runValidation` (system), `approvePromotion`
/ `rejectPromotion`, `activatePromotion` / `deactivatePromotion` / `archivePromotion`
- inne domeny: `approveOrder` / `cancelOrder`, `adjustStock`, `importProductsFromCsv` /
`exportProducts`, `toggleFeatureFlag`

Nazwy intencyjne zamiast generycznego `update()` — materiał do rozmowy o granicy między prostym CRUD a komendą procesową.

Co ćwiczymy na tym case study

Dzień	Fokus	Przykładowe ekrany
Dzień 1	analiza couplingu i kandydaci na granice	Dashboard, <code>/products</code> , <code>/promotions/new</code>
Dzień 2	modularność, ekstrakcja i kompozycja MFE	<code>/suppliers</code> , <code>/orders</code> , widget „Performance”
Dzień 3	security, observability, governance i rollout	<code>/approvals</code> , <code>/settings/*</code>

Drivery, cele i scenariusze
jakościowe

Drivery, cele i scenariusze jakościowe

- najpierw „dlaczego?”, potem „jak?”
- driver ≠ rozwiązanie technologiczne
- cele muszą być mierzalne — inaczej nie da się ich zweryfikować
- architektura to świadome zarządzanie kompromisami, nie pogoń za ideałem

Najpierw drivers, potem rozwiązania

Typowy błąd:

```
Chcemy Kubernetes. Chcemy Kafkę.  
Chcemy microservices. Chcemy event sourcing.
```

To są **rozwiązania**. Najpierw pytamy: *jaki problem próbujemy rozwiązać?*

Rozwiązanie: Potrzebujemy Kafki. **Driver:** Musimy obsługiwać 20 000 zdarzeń/s i pozwalać kilku systemom niezależnie konsumować historię zdarzeń.

Jeden driver → wiele możliwych odpowiedzi: cache, CDN, batching, read replicas, partycjonowanie, kolejka, przetwarzanie asynchroniczne, skalowanie poziome, denormalizacja...

```
driver ≠ solution
```

Czym jest driver architektoniczny?

Architectural driver to czynnik mający istotny wpływ na: strukturę systemu, odpowiedzialności komponentów, technologie, sposób integracji, deployment, sposób działania systemu.

Nie każde wymaganie jest driverem.

Wymaganie: Użytkownik może zmienić nazwę profilu. — prawdopodobnie niewielki wpływ architektoniczny.

Driver: System musi obsługiwać logowanie 5 mln użytkowników i działać podczas awarii pojedynczego regionu. — wpływa na wiele fundamentalnych decyzji.

Wymaganie, driver, constraint, assumption, risk

Pojęcie	Przykład	Reagujemy przez
Requirement	użytkownik może pobrać fakturę	projektowanie funkcjonalności
Driver	100 tys. faktur generowanych w godzinę	decyzje architektoniczne
Constraint	dane muszą pozostać w UE	zawężenie przestrzeni rozwiązań
Assumption	maks. 5 tys. aktywnych użytkowników	weryfikację założenia
Risk	integracja partnera może nie obsłużyć przepustowości	plan mitygacji

Constraint różni się od requirementu: przy requirement negocjujemy *sposób* realizacji (`dostępność 99,9%`), przy constraint przestrzeń rozwiązań jest już zamknięta (`deployment musi działać na Azure`). Warto dopytać: *czy to naprawdę ograniczenie, czy tylko dotychczasowy zwyczaj?*

„Musimy użyć X” — jakiego rodzaju to ograniczenie?

Przykład: *Musimy używać Oracle.*

Dopytaj: kto tak zdecydował? z jakiego powodu? czy wynika to z licencji, regulacji, kompetencji, istniejącej umowy, integracji, standardu organizacji?

Oracle wymagane przez regulatora	→ hard constraint
Oracle objęte kontraktem do 2029	→ contractual constraint
DBA znają tylko Oracle	→ organizational constraint
„Zawsze używaliśmy Oracle”	→ historical constraint / assumption

Hard constraint się nie zmieni. Reszta — być może tak, jeśli ktoś świadomie zdecyduje się ponieść koszt zmiany.

Skąd biorą się drivery?

Architekt nie zaczyna od technologii. Zaczyna od otoczenia systemu.

Business • Users • Domain • Organization
Technology • Operations • Security • Regulation
Legacy • Future

Business: *Black Friday generuje 15× normalnego ruchu* — wpływa na cache, skalowanie, model danych, proces checkout, integracje.

Domain: granice modułów e-commerce (Catalog • Pricing • Inventory • Orders • Payments) mają różne tempo zmian i różne wymagania jakościowe — struktura techniczna nie powinna ignorować struktury biznesu.

Legacy: Nowy system → REST → Adapter → SOAP → Legacy — adapter bywa ważniejszą decyzją niż wybór frameworka frontendowego.

Interesariusze — sprzeczne potrzeby

Interesariusz to osoba, która ma wpływ na projekt lub na którą projekt ma wpływ.

Interesariusz	Czego chce
użytkownik	prostoty
security	kontroli
biznes	szybkich zmian
operations	stabilności
developer	łatwości modyfikacji
finance	niskiego kosztu
regulator	audytowalności

Prawo Conwaya i organizacja jako część architektury

„Any organization that designs a system will produce a design whose structure is a copy of the organization's communication structure.”

Architektura wyznacza strukturę organizacji lub **na odwrót** — organizacja jest driverem, nie tylko kontekstem.

1 zespół → modular monolith bywa bardzo efektywny
20 niezależnych zespołów → niezależność wdrożeń staje się driverem

Funkcjonalność istotna architektonicznie

Nie każda funkcja ma ten sam wpływ na architekturę.

Zwykła funkcjonalność: użytkownik zmienia nazwę produktu.

Architecturally Significant Requirement (ASR): podczas zakupu system musi atomowo zarezerwować dostępny towar w 200 magazynach.

Drugi przypadek wpływa na: consistency, concurrency, integracje, model danych, availability, failure handling — to sygnał, że mamy do czynienia z driverem, nie tylko wymaganiem.

Atrybuty jakościowe jako mierzalny scenariusz

Rozróżnijmy pojęcia:

- **Functional requirement** — co system ma robić.
- **Quality attribute / architecture characteristic** — jak dobrze to robi (np. dostępność 99,9%).
- **„-ilities”** — potoczne określenie atrybutów jakościowych (scalability, testability, maintainability...).

Źle: *System musi być szybki.* Lepiej: *95% requestów poniżej 500 ms.* Jeszcze lepiej: *Przy 3000 równoczesnych użytkownikach 95% zapytań wyszukiwania kończy się poniżej 500 ms.*

Dopiero taki zapis można zaprojektować, przetestować i ocenić.

Anatomia Quality Attribute Scenario

Przydatny szablon zamiast hasła „system ma być X”:

```
Source      → kto/co wywołuje bodziec
Stimulus    → co się dzieje
Environment → w jakich warunkach
Artifact    → która część systemu
Response    → jak system reaguje
Response measure → jak to mierzymy
```

Każdy atrybut jakościowy warto przepuścić przez ten szablon, zanim trafi do listy wymagań.

Przykłady scenariuszy

Atrybut	Źle	Scenariusz
Performance	„system ma być szybki”	Black Friday, 3× ruchu → $p95 < 500 \text{ ms}$, $\text{error rate} < 0.5\%$
Availability	„system musi być highly available”	awaria 1 noda → niedostępność $< 30 \text{ s}$, $\text{RTO} < 30 \text{ s}$, $\text{RPO} = 0$
Modifiability	„system powinien być elastyczny”	nowy kurier → zmiany tylko w Shipping , $\leq 2 \text{ dni}$, bez migracji danych
Security	„system musi być bezpieczny”	użytkownik bez roli Finance nie odczyta danych płatności przez UI, API ani raporty

Konkretny scenariusz jest jednocześnie wymaganiem i kryterium testu.

Nie da się maksymalizować wszystkiego

Typowe napięcia:

Security ↔ Convenience
Consistency ↔ Availability
Performance ↔ Cost
Flexibility ↔ Simplicity
Time-to-market ↔ Future flexibility

Architektura nie usuwa kompromisów — czyni je świadomymi.

Kompromis	Opcja A	Opcja B
Security vs usability	MFA + hardware key + approval	użytkownicy współdzielą sesje, omijają proces
Consistency vs availability	strong consistency, wyższe latency	eventual consistency, chwilowo niespójny stan

Top 3–5 driverów i ranking

Lista 30 wymagań jakościowych nie pomaga podejmować decyzji. Pytanie: *które kilka rzeczy naprawdę kształtuje architekturę?*

1. Time-to-market
2. Integracja z legacy
3. Auditability
4. Peak scalability
5. Independent deployment

Driver	Importance	Difficulty/Risk
Availability	5	4
Time-to-market	5	2
Auditability	5	4
Modifiability	4	3

Dobra dyskusja o priorytetach

Zamiast: *Czy performance jest ważny?* — pytaj o konkretną konsekwencję:

- *Czy wybralibyśmy rozwiązanie dwukrotnie droższe, jeśli obniży latency z 500 ms do 200 ms?*
- *Czy zaakceptujemy 30 minut niedostępności raz w roku?*
- *Czy niezależne wdrożenie modułu jest warte dodatkowej złożoności operacyjnej?*

„Wymaganie” bywa dopiero **początkiem rozmowy**: *Wszystko musi działać 24/7* → po dopytaniu:

Checkout:	99.99%
Order history:	99.9%
Reporting:	99.0%
Admin panel:	business hours

Koszt architektury może spaść dramatycznie.

Driver → decision → consequence

Dokumentujemy nie tylko „wybraliśmy X”, ale dlaczego i z jakim kosztem.

```
Driver: 5 niezależnych zespołów
↓
Decision: oddzielne bounded contexts
↓
Consequence: jasna własność kodu i danych
+ większy koszt integracji
```

Jedna decyzja może obsługiwać kilka driverów naraz — np. asynchronous messaging pomaga w scalability, availability, loose coupling, ale pogarsza consistency, debugging i latency. Analiza tych konsekwencji to właściwa praca architektoniczna.

Antywzorce przy formułowaniu driverów

Architecture by technology: Mamy Kubernetes, Kafkę, GraphQL, microservices — brakuje pytania *dłaczego?*

Architecture by fashion: Netflix używa microservices, Uber używa Kafki — brakuje informacji o skali, organizacji, budżecie i problemie, który tamte firmy faktycznie rozwiązują.

„Wszystkie quality attributes są ważne”: Fast • Secure • Scalable • Maintainable • Cheap • Simple — nie mówi nic. Przydatna informacja zaczyna się od: które? jak bardzo? co poświęcimy w zamian?

Case: system zamówień B2B

Założenia: backoffice B2B, 500 firm-klientów, 20 000 zamówień dziennie, integracja z ERP, częste zmiany cenników, część klientów wymaga indywidualnych reguł, zespół 12 developerów, wdrożenia kilka razy w tygodniu.

Pytanie: jakich driverów szukamy? (business, skala, domena, integracja, organizacja, security)

Przykładowy Top 5:

1. Integracja z niestabilnym ERP
2. Szybkie zmiany reguł biznesowych
3. Tenant isolation
4. Wdrażanie kilka razy w tygodniu
5. Obsługa 5× peak traffic

Dopiero teraz zaczyna się dyskusja o granicach modułów, synchroniczności, cache i danych.

Drivery a cele architektury

Driver → Cel → Decyzja / Struktura systemu

- Driver opisuje presję, potrzebę lub ograniczenie.
- Cel opisuje oczekiwany rezultat odpowiedzi na te czynniki.
- Decyzje i struktura systemu powinny dać się powiązać z konkretnym celem.

Podstawowe cele architektury: wsparcie celów biznesowych, zapewnienie wymaganych właściwości jakościowych, ograniczenie najważniejszych ryzyk, bezpieczny rozwój systemu, ułatwienie pracy wielu zespołów, kontrola kosztów, zgodność z regulacjami, przygotowanie na prawdopodobne (nie wszystkie wyobrażalne) zmiany.

Formułowanie dobrych celów i non-goals

Dobry cel: powiązany z driverem i interesariuszem, ma określony zakres i horyzont czasowy, ma mierzalne kryterium sukcesu, oddziela cel od wybranego rozwiązania.

Non-goals — jawne określenie, czego architektura **nie** próbuje osiągnąć.

Równie ważne jak lista celów: chroni przed niekończącym się rozszerzaniem zakresu i projektowaniem pod problemy, których nie mamy.

Architektura jako zarządzanie kompromisami

Nie istnieje architektura najlepsza bez kontekstu — każde rozwiązanie wprowadza korzyści, koszty i nowe ryzyka. „Dobra architektura” jest wystarczająca dla danych driverów, nie idealna w próżni.

- Atrybuty jakości osiągamy przez sprawdzone wzorce; style to zestawy wzorców, które dobrze ze sobą współpracują.
- Decyzje powinny pozostawać odwracalne tam, gdzie to możliwe — nadmierna abstrakcja „na zapas” to częste źródło kosztu.



Ocena realizacji celów

- Kryteria i testy: akceptacja decyzji architektonicznych, testy wydajnościowe, bezpieczeństwa, odporności.
- Miary architektoniczne: czy architektura wspiera strategię firmy (KPI/OKR), jaki ma wpływ na koszt, harmonogram i strukturę organizacji.
- Poziomy wartości IT: centrum kosztów → część strategii firmy → katalizator strategii (Porter Value Chain).
- Fitness functions: automatyczna, powtarzalna kontrola reguł architektonicznych (np. testy zależności między modułami w CI) — weryfikacja na



Priorytetyzacja Must / Target / Stretch

Nie wszystkie cele mają tę samą wagę.

- **Must:** bez tego rozwiązanie odpada — dane klientów nie mogą opuścić UE
- **Target:** poziom oczekiwany — $p95 < 500$ ms
- **Stretch:** ambitny, ale opcjonalny — $p95 < 200$ ms

To upraszcza decyzje przy konflikcie koszt vs jakość i chroni przed projektowaniem „najbardziej skalowalnego, najbezpieczniejszego i najszybszego systemu świata”.



Podsumowanie

Context → Drivers → Priorities → Trade-offs
→ Decisions → Architecture → Runtime feedback
→ New drivers ↻

- najpierw pytamy **dlaczego**, potem **jak**
- driver ≠ rozwiązanie, jeden driver może mieć wiele odpowiedzi
- quality attributes powinny być mierzalnymi scenariuszami (Source/Stimulus/.../Measure)
- nie da się maksymalizować wszystkiego — architektura to zarządzanie kompromisami
- Must/Target/Stretch pomaga rozstrzygać konflikty priorytetów
- drivers i cele zmieniają się wraz z systemem i organizacją

Dokumentacja architektury: po
co i dla kogo

Dokumentacja architektury: po co i dla kogo

- dokumentacja nie jest celem samym w sobie — służy decyzjom i komunikacji
- bez odbiorcy i celu to koszt, nie aktywo
- jeden dokument nie odpowie na wszystkie pytania — potrzeba wielu perspektyw
- brak ownera dokumentu = szybka dezaktualizacja

Dla kogo dokumentujemy

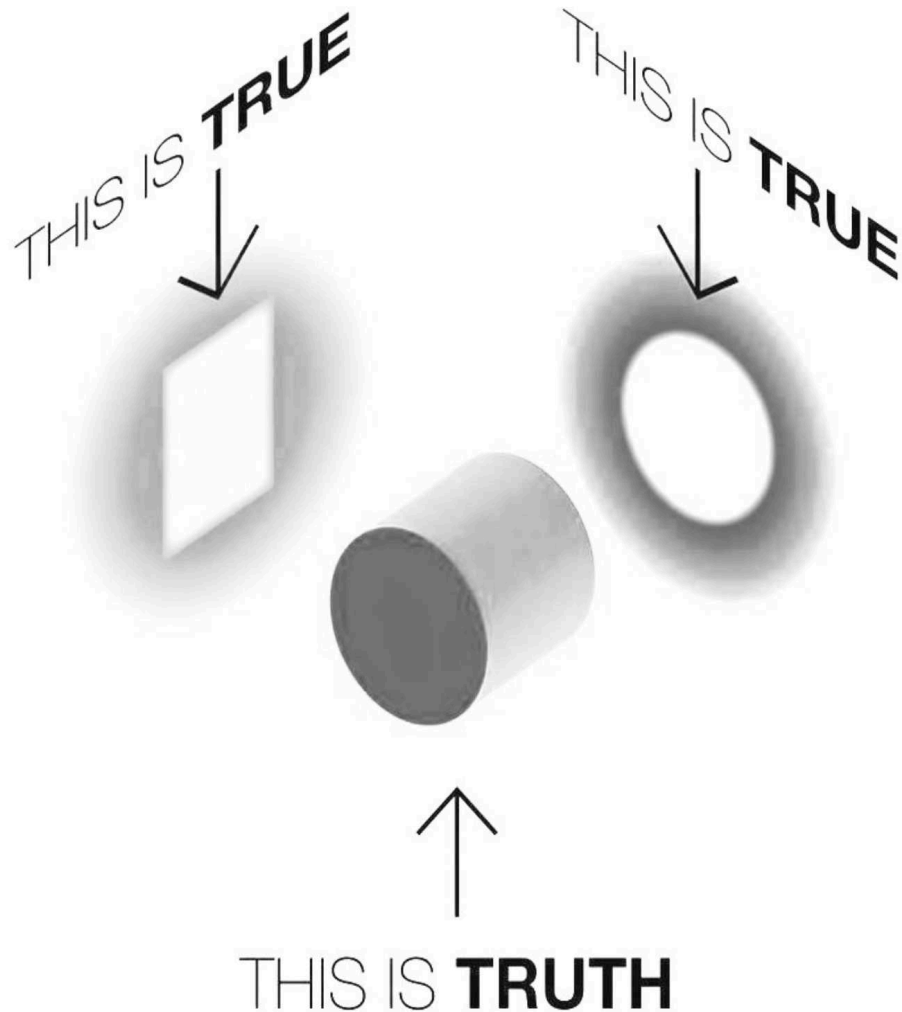
Odbiorca	Czego oczekuje
Biznes	cele, ryzyka, koszty
Deweloperzy	struktura, kontrakty, konwencje
Operacje	wdrożenie, monitoring, incydenty
Bezpieczeństwo	granice zaufania, dane wrażliwe
Przyszli członkowie zespołu	kontekst i historia decyzji

Ten sam system wymaga różnych dokumentów dla różnych odbiorców

Perspektywa, model, widok

- Model — złożona abstrakcja fragmentu systemu
- Widok — pokazuje model z konkretnej strony
- Perspektywa — widok z punktu widzenia konkretnego interesariusza

Architekci projektują modele, a dopiero potem dokumentują ich widoki — widok jest reprezentacją struktury modelu, nie odwrotnie



Jakie widoki

- kontekst systemu, funkcjonalny, logiczny, fizyczny
- deployment, development, informacyjny, procesowy
- bezpieczeństwa, wydajność i skalowalność, operacyjny
- konkurencji, dostępność i niezawodność, regulacyjny

Każdy widok odpowiada na inne pytanie i jest przeznaczony dla innego odbiorcy — nie próbujemy zmieścić wszystkiego w jednym

Model 4+1

Cztery widoki plus scenariusze jako element spinający całość (Kruchten, 1995):

Widok	Odpowiada na pytanie
Logiczny	jakie są kluczowe abstrakcje i relacje między nimi?
Procesów	jak wygląda współbieżność i komunikacja w runtime?
Fizyczny	jak system mapuje się na infrastrukturę?
Wdrożenia	jak zbudować, spakować i uruchomić system?



Wiele perspektyw zamiast jednego „kompletnego diagramu”

- nie da się opisać architektury jednym, wyczerpującym diagramem
- każda perspektywa odpowiada na inne pytanie innego odbiorcy
- diagram „na wszystko” zwykle nie odpowiada dobrze na nic

Stan obecny, stan docelowy, historia decyzji

Dokumentacja architektoniczna miesza trzy różne osie czasu:

- Stan obecny (AS-IS) — jak system działa dzisiaj
- Stan docelowy (TO-BE) — dokąd zmierzamy
- Historia decyzji — dlaczego jest tak, jak jest

Brak rozróżnienia tych trzech osi to częsta przyczyna nieporozumień

Minimalna dokumentacja wystarczająca do zadania

- dokumentuj tyle, ile potrzeba do wykonania konkretnego zadania — nie więcej
- nadmiar dokumentacji jest równie kosztowny co jej brak
- pytanie kontrolne: *„kto i do czego użyje tego dokumentu?”*

Dokumentacja żyjąca razem z systemem

- dokumentacja bez właściciela szybko traci aktualność
- każdy dokument ma właściciela i datę ostatniej aktualizacji
- wersjonowana razem z kodem, nie obok niego
- nieaktualna dokumentacja jest gorsza niż jej brak — świadomie myli czytelnika

Dokumentacja jako kod

- markdown w repo + pull request
- przegląd dokumentu razem z kodem, w tym samym code review
- śledzenie historii zmian i decyzji przez git
- automatyczna walidacja linków/diagramów, publikacja jako strona statyczna

Minimalny łańcuch artefaktów



Każdy artefakt powinien dać się powiązać z poprzednim i następnym ogniwem tego łańcucha

Checklist „wystarczająco dobrej” dokumentacji

- kto jest odbiorcą?
- jaka decyzja ma z tego wyniknąć?
- co jest AS-IS, a co TO-BE?
- gdzie jest owner i data aktualizacji?

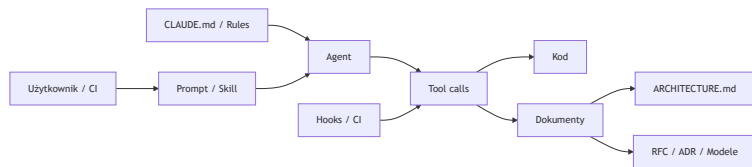
Architektura z AI: workflow

Architektura z AI: workflow

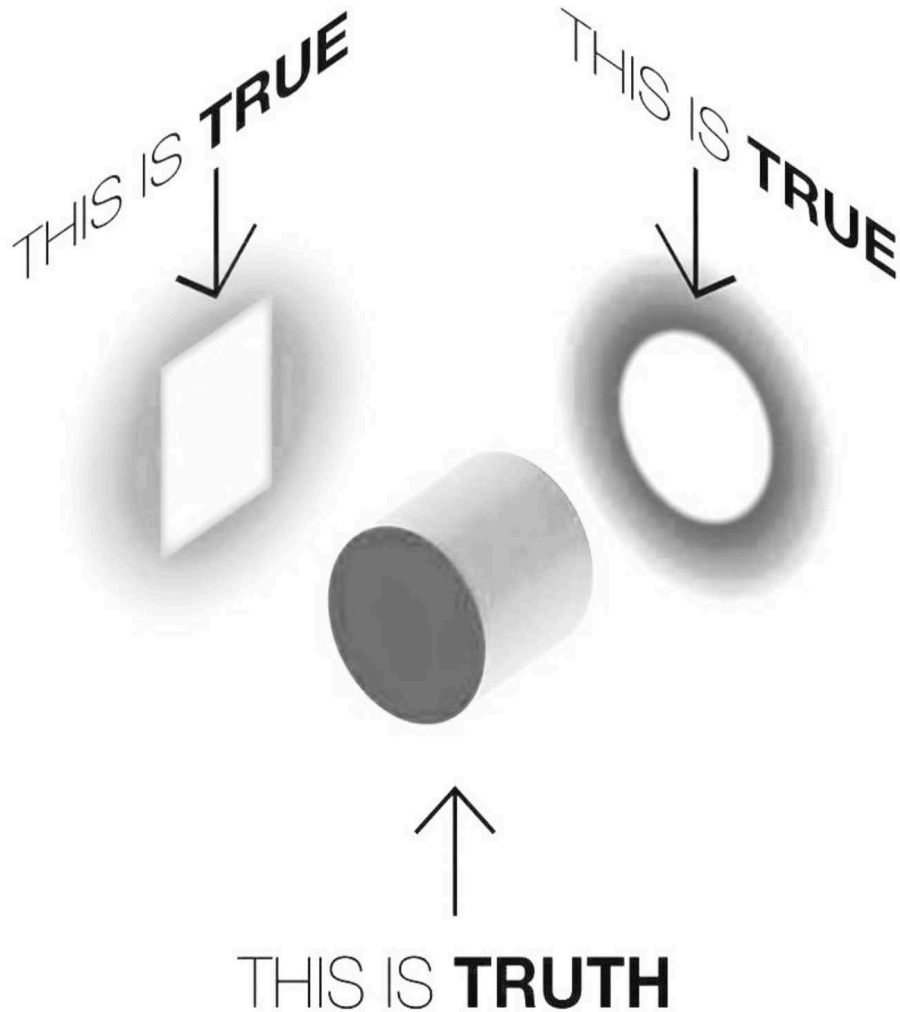
- AI przyspiesza pętlę decyzja → dokumentacja → weryfikacja
- pomaga odkrywać architekturę, prowadzić analizy, tworzyć diagramy i drafty
- prompt / instrukcja / skill / agent — różne poziomy trwałości
- checkpoint i akceptacja człowieka zostają obowiązkowe
- AI nie zastępuje właściciela decyzji

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width,
    <title>Document</title>
  </head>
  <body>
    <div class="container">
      <div class="row">
        <div class="col">
          <h1>Welcome</h1>
          <p>
            Lorem ipsum dolor sit amet consectetur adip
            laborum, placeat ad labore impedit aspernat
            vero qui, rerum, accusamus eos incidunt, de
            asperiores!
          </p>
        </div>
      </div>
    </div>
    <script src="scripts/main.js"></script>
    <script src="scripts/plugins.js"></script>
  </body>
</html>
```

Jedno źródło prawdy, wiele mechanizmów



CLAUDE.md, prompt i agent nie przechowują kopii architektury — kierują do właściwych źródeł i uruchamiają proces.



Warstwy pracy z AI

Warstwa	Mechanizmy	Odpowiada na pytanie
Kontekst	CLAUDE.md , AGENTS.md , rules	Co agent powinien zawsze wiedzieć?
Workflow	prompt, skill	Jak wykonać powtarzalne zadanie?
Wykonanie	agent, tool call, MCP	Kto i czym wykona działanie?
Egzekwowanie	hook, skrypt, CI	Co musi wydarzyć się zawsze?
Dystrybucja	plugin	Jak udostępnić całość innym?

Każda warstwa ma inną odpowiedzialność — nie zastępują się wzajemnie.

Prompt, instrukcja czy skill?

	Prompt	Instrukcja / rule	Skill
Co to	jednorazowe / zapisane zadanie	stała lub warunkowa zasada	wiedza + wieloetapowy workflow
Kiedy działa	ręcznie, jeden rezultat	zawsze albo pod warunkiem	ładowany na żądanie
Przykład	„wygeneruj C4 Container”	„oznaczaj AS-IS i TO-BE”	analiza architektury, prowadzenie RFC
Zawiera	krótki opis celu	standard / ograniczenie	checklisty, szablony, skrypty

Reguła wyboru

powtarzam polecenie → prompt · powtarzam procedurę → skill

CLAUDE.md , AGENTS.md , ARCHITECTURE.md

- CLAUDE.md — krótki, zawsze ładowany kontekst Claude Code
- AGENTS.md — wspólne instrukcje dla wielu narzędzi AI
- ARCHITECTURE.md — żywa dokumentacja systemu dla ludzi i agentów

CLAUDE.md może importować @AGENTS.md i wskazywać kiedy przeczytać architekturę.

Router ≠ źródło prawdy

Nie kopiuj pełnej architektury do instrukcji agenta — instrukcja ma kierować, nie duplikować wiedzy.

Agent, subagent, tool call, MCP, hook

- Custom agent — wyspecjalizowana rola, własna instrukcja, model i lista tools, może działać przez całą sesję (np. `domain-architect`)
- Subagent — osobne okno kontekstu, wykonuje delegowane zadanie i zwraca tylko wynik; dobry dla dużych analiz i logów, nie do rozmowy z użytkownikiem
- Tool / tool call — tool to dostępna zdolność (odczyt, zapis, terminal), tool call to jedno jej użycie
- MCP — dostarcza tools i dane z zewnętrznych systemów (GitHub, Jira, wiki, katalog usług)
- Hook — uruchamia się przy zdarzeniu cyklu życia, waliduje/blokuje/loguje niezależnie od pamięci modelu

Skill dostarcza procedurę. Agent dostarcza rolę i możliwości.

Docelowa struktura repo

```
CLAUDE.md
AGENTS.md
.claude/
  rules/
  skills/
  agents/
  hooks/
.github/
  prompts/
docs/
  ARCHITECTURE.md
  adr/
  rfc/
  rules/
  diagrams/
  reports/
```

Pliki `.claude/skills/`, `.claude/rules/` i `.claude/agents/` mogą stanowić wspólny rdzeń dla Claude Code i VS Code.

Tworzenie narzędzi procesu

Od powtarzanego prompta do wersjonowanego workflow

VS Code — `/create-prompt` (zapisane zadanie), `/create-skill` (workflow + zasoby), `/create-instruction` (warunkowa reguła), `/create-agent` (rola + tools), `/init` (instrukcje workspace)

Claude Code — wystarczy opisać pliki i zachowanie:

```
Utwórz project skill `managing-rfc` w `.claude/skills/managing-rfc/`.  
Dodaj szablon, checklistę jakości i skrypt numerujący RFC.
```

`/skills` (lista) · `/reload-skills` (przeładowanie) · `plugin validate` (walidacja)

Nie duplikuj checklisty w czterech promptach — wywołuj jeden skill.

/project-help — wejście do projektu

Pomaga nowej osobie lub agentowi:

- zrozumieć cel projektu i znaleźć aplikacje/dokumenty
- odpowiedzieć na pytanie na podstawie repo
- wskazać kod, ADR, RFC albo diagram jako dowód
- jawnie wypisać niepewności

ARCHITECTURE.md → indeksy → README → manifesty → kod

Tryb read-only. Nazwa /project-help nie koliduje z wbudowanym /help .

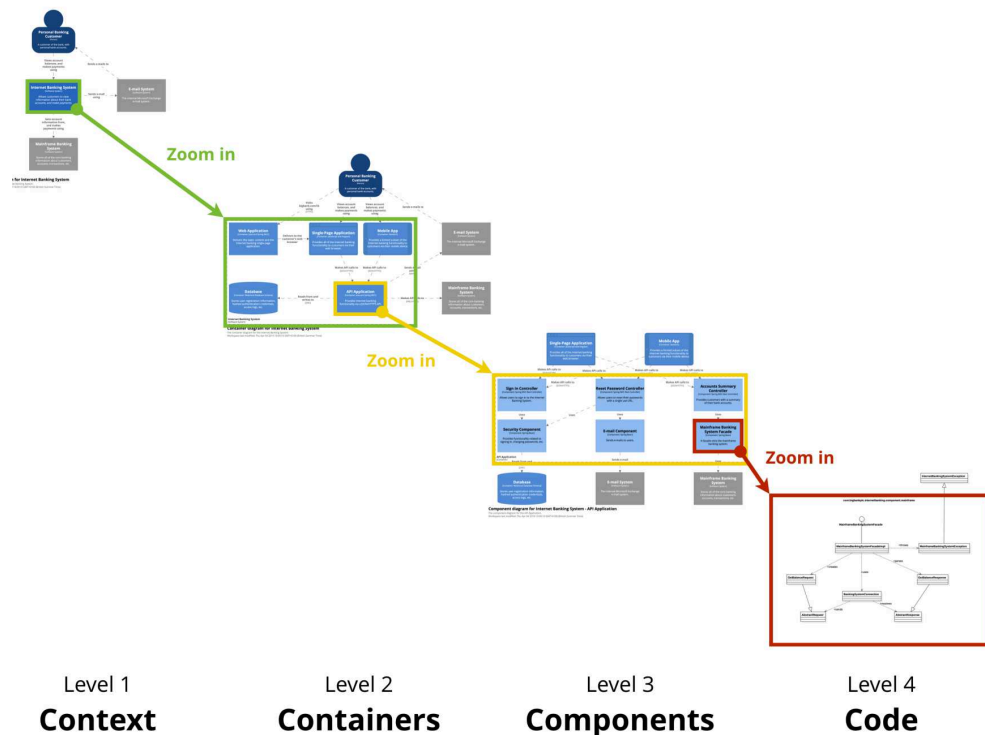
Diagramy architektury na żądanie

Jeden skill, cztery komendy — C4 i 4+1 bez ręcznego pisania prompta:

```
/generating-architecture-diagrams  
c4-system | c4-container | c4-component | four-pl  
  
<typ> <zakres> <stan> <plik-wyjściowy>
```

Domyślnie: zakres z `ARCHITECTURE.md`, stan `AS-IS`, format Markdown + Mermaid, brak danych → `TBD` (bez zgadywania), zapis w `docs/diagrams/`. Każdy dokument ma odbiorcę, datę, zakres, wyłączenia i źródła.

Nie przenoś elementu z TO-BE do AS-IS bez dowodu w kodzie lub



`/init` ≠ analiza architektury

`/init` — tworzy początkowy `CLAUDE.md`, odkrywa komendy i konwencje, przygotowuje instrukcję pracy agenta.

`/analyzing-architecture` — bada rzeczywistą strukturę systemu, rozdziela fakty/wnioski/hipotezy, tworzy raport, zapisuje niepewności jako pytania, nie zmienia kodu ani źródła prawdy. Domyślnie działa autonomicznie — bez pytań do użytkownika.

Checklista: kontekst · struktura · zależności · domeny · runtime · delivery · operacyjność · drift · ryzyka · rekomendacje

Raport AI (executive summary, AS-IS/TO-BE, ryzyka, rekomendacje z dowodami) opiera się na deterministycznym collectorze (manifesty, lockfile, skrypty, wyniki build/test/lint). Skrypt zbiera fakty, model interpretuje ich znaczenie.

RFC i analiza architektury z agentem

/managing-rfc — menu procesu:

```
new [tytuł] [ścieżka]    continue [plik]    quality [plik]
grill [plik]             open-questions    stakeholder-questions
```

Bez argumentów skill pokazuje menu. Numer nadaje skrypt, nowy dokument zaczyna ze statusem **Draft** w `docs/rfc/`.

new — problem, drivery, zakres, owner, AS-IS, alternatywy, rollout, rollback. **continue** — czyta ostatni draft i log sesji, wskazuje największą lukę.

Checklista **quality** : problem vs rozwiązanie · AS-IS z dowodami · zakres i non-goals · realne alternatywy · migracja/rollback · kryteria sukcesu · owner · gotowość na ADR → wynik **PASS** / **WARN** / **FAIL** .

grill — clarification session i checkpoint

1. Zadawaj jedno pytanie naraz, wynikające z odpowiedzi albo luki w RFC
2. Nie odpowiadaj za użytkownika
3. Co 5 odpowiedzi utwórz **checkpoint** (pytania, fakty, decyzje robocze, sprzeczności, zmiany w RFC)
4. Po checkpointie wróć do największej niewiadomej
5. **stop** / **koniec** zapisuje podsumowanie i kończy sesję

Tryb działa w głównej rozmowie — subagent nie prowadzi dialogu z użytkownikiem. Człowiek zatwierdza status **Accepted**, granice, ownership i deployment.

Agent przygotowuje decyzję. Właściciel ją podejmuje.

Agent Microfrontend / Monorepo

Co potrafi analizować — npm/pnpm/yarn workspaces, Nx i Turborepo, graf zależności i cykle, naruszenia granic, hosty i remote'y, `exposes / remotes / shared`, routing/state/kontrakty, coupling deploymentowy i organizacyjny.

Co może uruchamiać — build, test, lint, e2e, dependency graph, dev server na żądanie, istniejące code generators, Sheriff / dependency-cruiser / boundaries, raport driftu.

Tylko narzędzia wykryte jako zainstalowane i skonfigurowane — bez zakładania nazw skryptów.

Najpierw wykryj toolchain, potem tryb

Lockfile + packageManager → package.json → Workspace?

Nx → nx.json + targets

Turbo → turbo.json + tasks

npm/pnpm → workspaces + scripts

Tryb	Zakres
<code>inspect</code>	tylko analiza — domyślny
<code>quality</code> / <code>boundaries</code>	build/test/lint, graf i naruszenia
<code>run</code>	kontrolowany dev server
<code>generate</code> / <code>migrate</code>	wymaga planu i zatwierdzenia

Guardrails: zakaz publish/deploy/push bez polecenia, minimalny gate → potem pełna walidacja.

Automatyzacja i governance

Model rozumuje — skrypt mierzy — hook egzekwuje

Zdarzenie	Automatyczna reakcja
PreToolUse: Edit Write	blokada zapisu poza dozwolonym zakresem
PreToolUse: terminal	blokada operacji destrukcyjnych / publish
PostToolUse	format i lekka walidacja dokumentu
Stop / TaskCompleted	gate przed zamknięciem zadania
ConfigChange	audyt zmian skills, rules i settings

Hook uruchamia się zawsze dla zdarzenia — nie liczy na pamięć modelu.

Skrypt czy AI? Dwa pipeline'y CI

Skrypt — numeruje RFC/ADR, waliduje frontmatter i sekcje, sprawdza linki i Mermaid, pilnuje immutable ADR.

AI — ocenia argumentację, szuka niejawnych założeń, proponuje alternatywy, prowadzi sesję clarification.

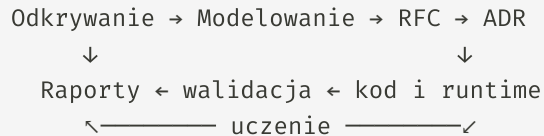
Obowiązkowy pipeline (deterministyczny): frontmatter, sekcje, linki, indeksy, dependency drift. **Opcjonalny pipeline (semantyczny AI):** read-only na diffie, komentarz do review — **nigdy sam nie akceptuje decyzji.**

Weryfikacja skilla: happy path, braki i sprzeczności (TBD , bez halucynacji), naruszenie ograniczeń.

Guardrails i antywzorce

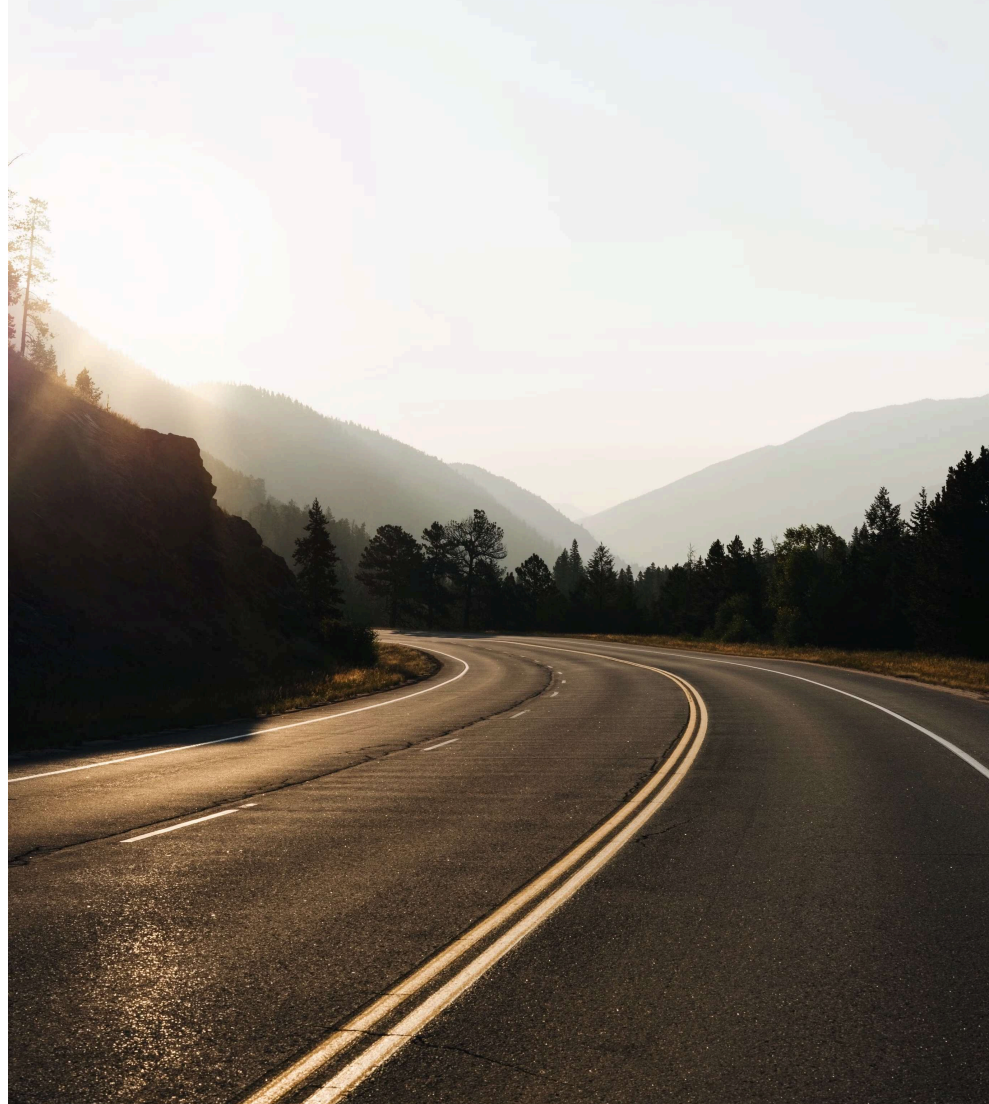
- agent proponuje, człowiek decyduje — status `Accepted` nigdy nadawany przez model
- automatyczna walidacja (fitness functions) w CI, ślad decyzji: driver → RFC → ADR → kod
- kopia `ARCHITECTURE.md` w `CLAUDE.md` ; wszystkie ADR-y w stałym kontekście
- jeden „super-skill” do wszystkiego; interaktywny workflow uruchomiony w izolowanym subagencji
- „nie zmieniaj kodu” zapisane tylko w instrukcji, bez twardej blokady (hooka)
- TO-BE przedstawione jako obecny system; LLM użyty zamiast prostego walidatora
- instalacja narzędzia lub masowy generator uruchomiony bez decyzji człowieka
- założenie, że dany tool albo npm script istnieje, bez wcześniejszej detekcji

Architektura z AI to proces



- `ARCHITECTURE.md` pozostaje źródłem prawdy
- skills kodują powtarzalne procedury, agenty izolują role i możliwości
- tools dostarczają fakty i wykonują działania
- hooks oraz CI egzekwują reguły — człowiek zachowuje odpowiedzialność za decyzje

AI przyspiesza pętlę — nie przejmuje ownershipu



Odkrywanie domen i Context Map

Odkrywanie domen i Context Map

- zaczynamy od procesu biznesowego, nie od endpointów
- granice domenowe są hipotezą do walidacji, nie ostateczną decyzją
- ten sam termin może znaczyć co innego w różnych kontekstach
- wynik warsztatu jest źródłem wiedzy do dalszej dokumentacji — nie finalną architekturą

Event Storming — metoda i trzy poziomy

Event Storming to lekka metoda warsztatowa do szybkiego eksplorowania tego, co dzieje się w domenie — celowo bez wsparcia komputerowego, na karteczkach samoprzylepnych na szerokiej ścianie.

- **Big Picture** — szerokie odkrywanie całej domeny, granic i hotspotów
- **Process Level** — szczegółowy przebieg konkretnego procesu biznesowego
- **Design Level** — poziom bliski projektowaniu agregatów i komend



Podstawowe elementy Event Stormingu

- Domain Events — fakty zapisane w czasie przeszłym, np. *Zamówienie złożone*, *Płatność otrzymana*
- Commands — intencje powodujące zmianę stanu, poprzedzają zdarzenie
- Actors — osoby lub systemy inicjujące komendy
- Policies — reguły typu „gdy zdarzenie X, uruchom komendę Y”
- Aggregates — granice spójności i odpowiedzialności
- Read Models — informacje potrzebne do podjęcia decyzji
- External Systems — systemy zewnętrzne uczestniczące w procesie
- Hotspots — pytania, konflikty i obszary wymagające dalszej analizy

Zdarzenia domenowe są niezmiennie, mają znacznik czasu i nie zawierają zachowania.

Przebieg sesji Event Stormingu

1. Chaotyczne zebranie zdarzeń domenowych (bez porządku)
2. Ułożenie zdarzeń na osi czasu
3. Identyfikacja alternatywnych ścieżek i wyjątków
4. Dodanie komend, aktorów i reguł
5. Ujednolicenie nazewnictwa
6. Wskazanie hotspotów i nieznanych informacji
7. Poszukiwanie granic procesów oraz modeli
8. Podsumowanie wyników i dalszych działań

Event Storming → hipotezy granic

Zdarzenia → Komendy → Aktorzy → Policies
→ Hotspoty → Kandydaci na bounded contexts

Nazwa zdarzenia z warsztatu często staje się nazwą zdarzenia integracyjnego (np. w kolejce) — to naturalne wejście do kontraktów między kontekstami.

Context Mapping i ograniczony kontekst

Projektowanie strategiczne polega na organizowaniu dużych systemów poprzez podział domeny na spójne ograniczone konteksty. Skupia się na granicach zespołów, spójności języka i integracji między podsystemami.

Ograniczony kontekst:

- posiada własny model, język i logikę
- ma jasne punkty integracji z innymi kontekstami
- ściśle odpowiada zespołowi lub poddomenie

Przykłady (e-commerce): kontekst wysyłki, kontekst rozliczeń, kontekst katalogu.

Oddzielne konteksty, wspólne pojęcia

Ten sam termin może mieć różne znaczenie w różnych kontekstach (martinfowler.com/bliki/BoundedContext).

- oba konteksty mogą mieć „Klienta” i „Produkt”, ale każdy z innymi właściwościami
- mogą dzielić ten sam identyfikator, by reprezentować dwa konteksty tego samego klienta
- reszta struktury i danych należy wyłącznie do danego kontekstu

Odpowiedzialność biznesowa i granice zespołów

- Oddzielne drogi (Separate Ways) — jeśli integracja jest zbyt kosztowna (technicznie lub politycznie), można ją pominąć; decyzję dokumentujemy w Mapie kontekstów
- Partnerstwo (Partnership) — zespoły o wspólnych celach koordynują harmonogramy wydań i wspólne komponenty
- Upstream / Downstream — kontekst downstream zależy od danych, API lub zachowania upstream; relacja wpływa na komunikację i planowanie, zwykle formalizowana kontraktem

Mapa kontekstów — wzorce integracji

Mapa kontekstów definiuje relacje i wzorce integracji między ograniczonymi kontekstami.

Wzorzec	W skrócie
Shared Kernel	wspólny podzbiór modelu, skoordynowane zmiany
Customer-Supplier	Klient zależny od Dostawcy, ale ma na niego wpływ
Conformist	Klient dostosowuje się do modelu Dostawcy bez zmian
Anticorruption Layer	warstwa tłumacząca, chroni model przed obcą logiką
Open Host Service	dobrze zdefiniowane API dla wielu konsumentów
Published Language	wspólny, uzgodniony format wymiany (schemat, kontrakt)

Przykład: kontekst rozliczeń jest Klientem kontekstu katalogu (Dostawca); kontekst UI używa ACL do tłumaczenia wywołań do modelu rozliczeń.

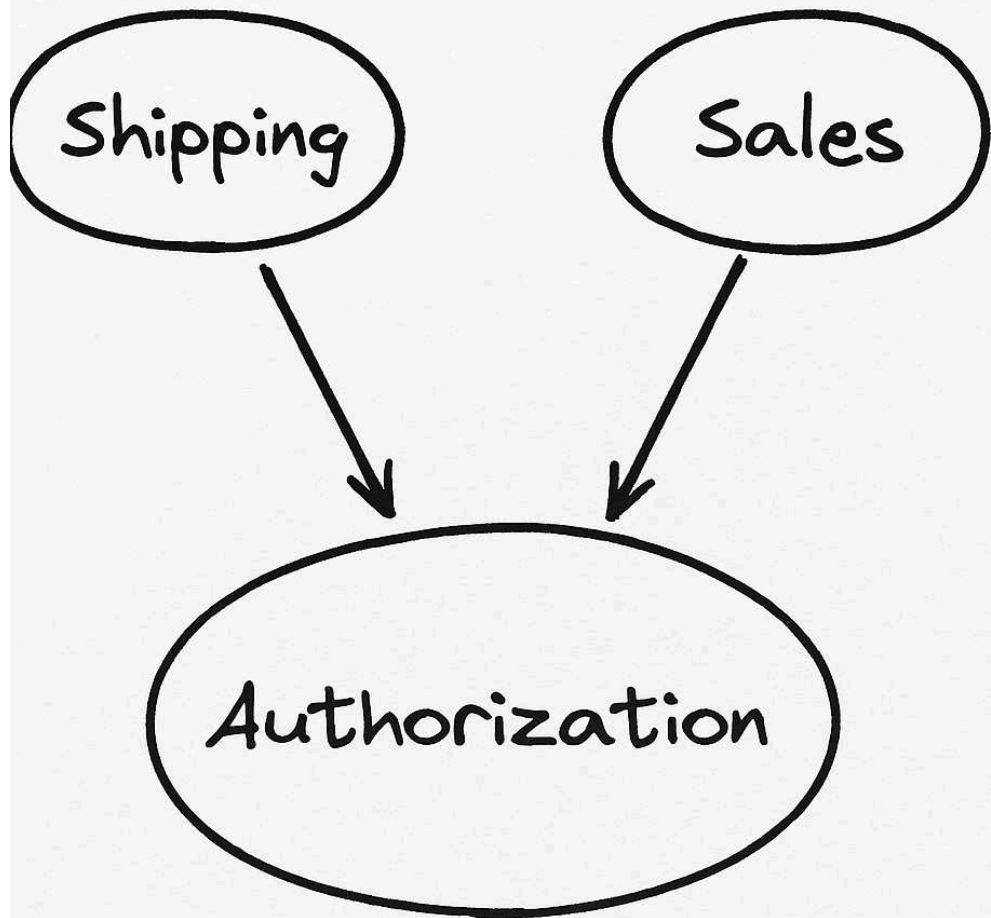
Shared Kernel i Customer-Supplier

Shared Kernel (wspólny rdzeń)

Dwa konteksty dzielą podzbiór modelu domeny i muszą koordynować zmiany. Używaj ostrożnie — silne powiązanie. *Przykład:* moduł uwierzytelniania współdzielony między Profilem użytkownika a Panelem administratora.

Customer-Supplier (Klient / Dostawca)

Jeden kontekst (Klient) zależy od drugiego (Dostawca) w zakresie funkcjonalności lub danych; Klient dostosowuje się do modelu i harmonogramu Dostawcy. *Przykład:* kontekst



Conformist i warstwa antykorupcyjna (ACL)

Conformist (Konformista)

Kontekst Klienta dostosowuje się do modelu Dostawcy bez zmian — szybka integracja, mała kontrola.

Przykład: system raportowania konsumujący surowe dane ze sprzedaży bez przekształcania ich.

Anticorruption Layer (ACL)

Warstwa tłumacząca między modelami, chroni integralność ograniczonego kontekstu — zasada Hollywood: „nie dzwoń do nas, my zadzwonimy do ciebie”. *Przykład:* CRM integruje się z ERP przez ACL, które tłumaczy „Klienta” ERP na „Klienta” CRM.

Usługa otwartego hosta i opublikowany język

- Open Host Service — udostępnia dobrze zdefiniowane API innym kontekstom
- Published Language — wspólny, uzgodniony protokół lub schemat

Przykład: kontekst wysyłki udostępnia REST API i używa specyfikacji OpenAPI (lub Protobuf) jako opublikowanego języka.

Core, Supporting, Generic Subdomains

Podział domeny wspiera priorytetyzację wysiłku zespołu:

- **Core domain** — odróżnia firmę od konkurencji; tu warto inwestować własny zespół
- **Supporting subdomain** — potrzebne, ale niekluczowe dla przewagi konkurencyjnej
- **Generic subdomain** — rozwiązany problem; dobry kandydat do kupienia lub outsourcingu (np. płatności, autentykacja)

Context Map dla Retail Operations Portal

Kontekst	Relacja	Wzorzec
Promotions	Customer of Catalog/Inventory	Customer-Supplier
Approvals	Shared Kernel z Identity (role)	Shared Kernel
Reporting	Conformist wobec zdarzeń domenowych	Conformist
Catalog	Open Host Service dla Promotions i Reporting	OHS + Published Language
Legacy ERP integration	Tłumaczenie modelu zamówień	Anticorruption Layer

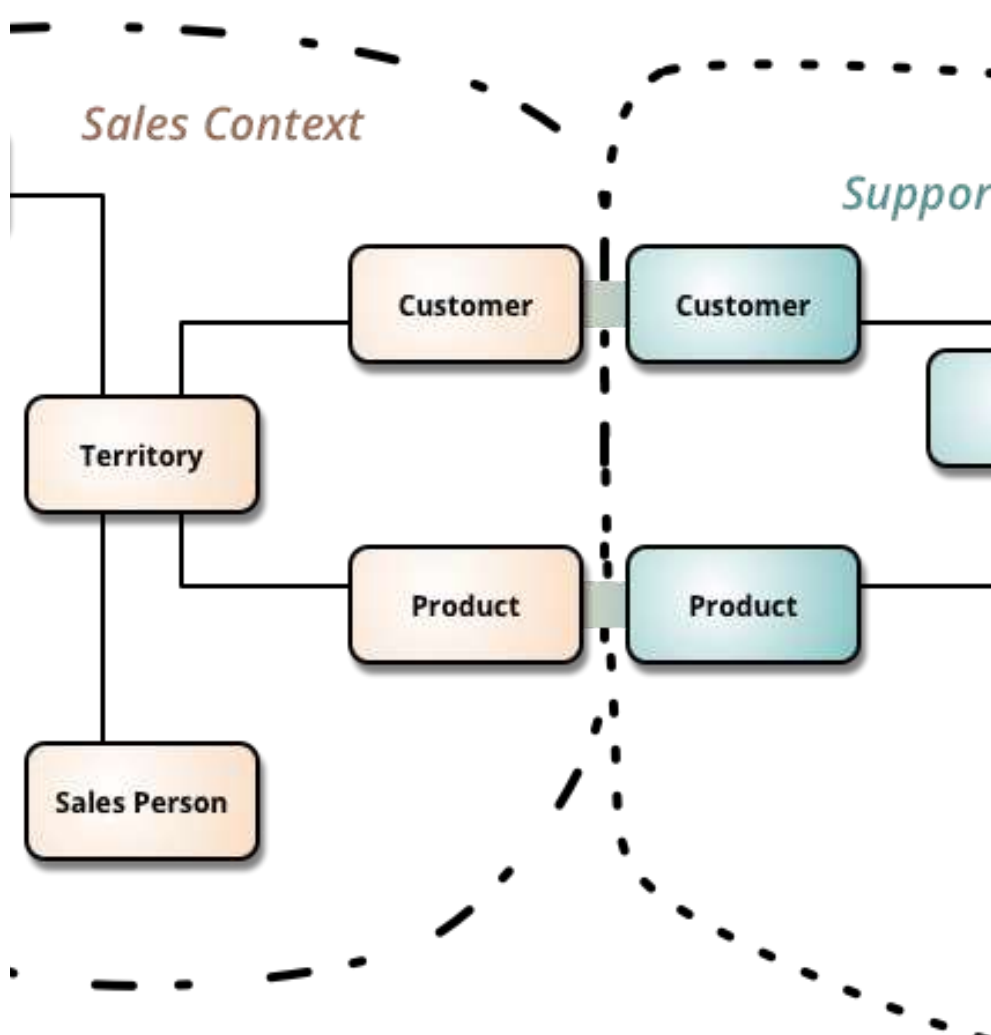
Catalog i Inventory to Core domain — tu inwestujemy najwięcej zespołu. Identity i Reporting to Generic/Supporting — kandydaci do gotowych rozwiązań lub minimalnej inwestycji.

Event Storming i Context Mapping z AI

Agent `domain-architect` (ze skillami `discovering-domains`, `facilitating-event-storming`, `mapping-contexts`) prowadzi warsztat i analizuje repozytorium — nie zmienia kodu aplikacji.

- `interactive` — jedno pytanie naraz, ekspert domenowy w pętli, checkpointy; wynik oznaczony jako `Reviewed` po akceptacji
- `analize` — czyta kod i dokumenty, nie zadaje pytań, oddziela fakty od interpretacji; wynik oznaczony jako `Hypothesis`

Event Storming → hipotezy granic
→ Context Map → RFC → ADR



Typowe błędy

- Rozpoczynanie od tabel, endpointów i mikroserwisów zamiast od domeny
- Zapisywanie operacji technicznych zamiast zdarzeń biznesowych
- Traktowanie pierwszej wersji modelu jako ostatecznej
- Brak ekspertów domenowych podczas warsztatu
- Mylenie granicy bounded context z granicą wdrożenia
- Automatyczne przekształcanie każdego kontekstu w osobny mikroserwis
- Zakończenie warsztatu bez listy pytań, decyzji i dalszych działań

Mini-lab: granice i ownership

1. Rozdziel: Products , Promotions , Approvals
2. Dla każdej granicy wskaż ownera (zespół, ekspert domenowy)
3. Nazwij wzorzec relacji między dwiema sąsiednimi granicami
4. Zapisz 1 ryzyko integracyjne i 1 kontrakt
5. Wskaż 1 hotspot, który wymaga dalszego Event Stormingu

C4, RFC i ADR: trzy narzędzia
jednej decyzji

C4, RFC i ADR: trzy narzędzia jednej decyzji

Trzy artefakty, jeden przepływ pracy nad decyzją architektoniczną:

Artefakt	Odpowiada na pytanie	Kiedy powstaje
C4	Jak system jest zbudowany?	Na bieżąco — stan obecny lub docelowy
RFC	Co proponujemy zmienić i dlaczego?	Przed podjęciem decyzji
ADR	Co ostatecznie wybraliśmy i dlaczego?	W momencie/po podjęciu decyzji

Modele opisują **strukturę**, RFC opisuje **propozycję**, ADR zapisuje **wynik**.

ADR — po co zapisywać decyzje

- Zapisujemy decyzje istotne architektonicznie — takie, które trudno odwrócić
- Kontekst i uzasadnienie znikają z pamięci zespołu szybciej, niż nam się wydaje
- Za pół roku nikt nie pamięta, *dla czego* wybrano dane rozwiązanie — tylko *co* wybrano
- ADR to najtańsza forma transferu wiedzy do przyszłego zespołu (włącznie z nami samymi za pół roku)

ADR a dokumentacja opisowa

	Opisuje	Odpowiada na pytanie
ADR	Decyzję i jej uzasadnienie	<i>Dlaczego system wygląda tak, a nie inaczej?</i>
C4 / dokumentacja opisowa	Aktualną strukturę systemu	<i>Jak system jest zbudowany?</i>

ADR-y i modele architektury uzupełniają się — nie zastępują.

Kiedy decyzja zasługuje na ADR? Gdy jest trudna lub kosztowna do odwrócenia, dotyczy wielu zespołów, wprowadza istotny trade-off albo ogranicza przyszłe opcje projektowe. Jeden ADR = jedna spójna decyzja.

Struktura ADR

```
# 12. Tytuł decyzji

Status: proposed

## Kontekst i problem
## Drivery i ograniczenia
## Rozważane warianty
## Decyzja
## Konsekwencje
### Pozytywne
### Negatywne
## Ryzyka i działania następce
```

- **Status** — proposed / accepted / rejected / deprecated / superseded
- **Konsekwencje** — plusy i minusy, nie tylko zalety wybranej opcji
- **Odnosniki** do RFC, diagramów, eksperymentów i poprzednich ADR

Cykl życia decyzji i rejestr ADR

- Propozycja → konsultacja → akceptacja
- Aktualizuj ADR, gdy doprecyzowujesz tę samą decyzję
- Utwórz nowy ADR, gdy zmieniasz samą decyzję — nie nadpisuj historii
- Stara decyzja dostaje status `superseded by ADR-XX` , ale **zostaje** w repozytorium
- Niektóre decyzje są świadomie **czasowe lub warunkowe** — wymagają ponownej oceny w określonym momencie
- Rejestr (indeks) ADR-ów pokazuje, które decyzje **wciąż obowiązują** — bez niego zespół działa na nieaktualnych ustaleniach

Praktyki zespołowe

- ADR przechowywany blisko kodu lub dokumentacji systemu (ten sam commit / PR)
- Przegląd ADR w ramach pull requestu — tak jak przegląd kodu
- Jasny właściciel decyzji i lista osób konsultowanych
- Powiązanie ADR z implementacją i zadaniami (link do PR, issue)
- Automatyczne generowanie indeksu decyzji z plików w repozytorium

Typowe błędy ADR

- Zapisywanie wyłącznie wyniku — bez kontekstu i uzasadnienia
- Przedstawianie wybranej opcji jako pozbawionej wad
- ADR tworzony po fakcie, wyłącznie żeby uzasadnić już podjętą decyzję
- Jeden rekord obejmujący kilka niezależnych decyzji naraz
- Usuwanie nieaktualnych ADR-ów zamiast oznaczenia `superseded` — niszczy historię
- Zapisywanie drobnych decyzji implementacyjnych — nadmiar dokumentacji zamiast sygnału

RFC — Request for Comments

RFC (propozycja)



dyskusja



decyzja(e)



ADR (wynik)

- RFC opisuje propozycję **przed** podjęciem decyzji, ADR zapisuje **wynik**
- Nie każde RFC kończy się akceptacją
- Jedno RFC może prowadzić do kilku ADR-ów
- Małe decyzje mogą wymagać tylko ADR-u, bez pełnego procesu RFC

Kiedy stosować RFC

- Zmiany obejmujące wiele zespołów lub systemów
- Nowe standardy, platformy albo współdzielone komponenty
- Zmiany kontraktów API, danych lub integracji
- Migracje i zmiany trudne do odwrócenia
- Rozwiązania wymagające konsultacji bezpieczeństwa, operacji albo biznesu
- Decyzje o znaczącym koszcie lub ryzyku

Struktura RFC

Podsumowanie i kontekst

- TL;DR, problem i motywacja, zakres oraz jawne **non-goals**, obecny stan systemu

Rozwiązanie i alternatywy

- Proponowane rozwiązanie, odrzucone alternatywy i powody ich odrzucenia
- Wpływ na API, dane, bezpieczeństwo i operacje

Wdrożenie i otwarte pytania

- Kompatybilność wsteczna i strategia migracji, strategia wdrożenia/testowania/rollbacku
- Obserwowalność i kryteria powodzenia, otwarte pytania i potrzebne eksperymenty

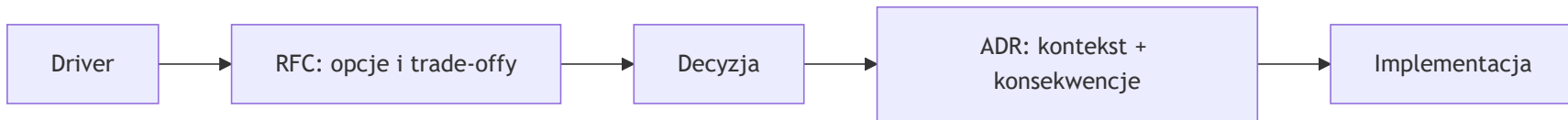
Proces konsultacji

- Autor przygotowuje RFC, właściciel podejmuje ostateczną decyzję
- Osoby wymagane do konsultacji vs osoby tylko informowane
- Jasny termin zbierania komentarzy — RFC bez deadline'u nie ma końca
- Komentarze blokujące vs nieblokujące
- Sprzeczne opinie rozstrzyga właściciel RFC, nie głosowanie
- Statusy: draft → in review → accepted / rejected / withdrawn → implemented

Typowe błędy RFC

- Przedstawienie tylko jednego wariantu jako oczywistego
- Konsultowanie rozwiązania, gdy implementacja jest już ukończona
- Brak osoby odpowiedzialnej za podjęcie ostatecznej decyzji
- Niekończące się zbieranie opinii — projektowanie przez komitet
- Nadmiernie ciężki proces dla niewielkich zmian
- Mieszanie dyskusji, decyzji i instrukcji implementacyjnej w jednym dokumencie

RFC → ADR flow

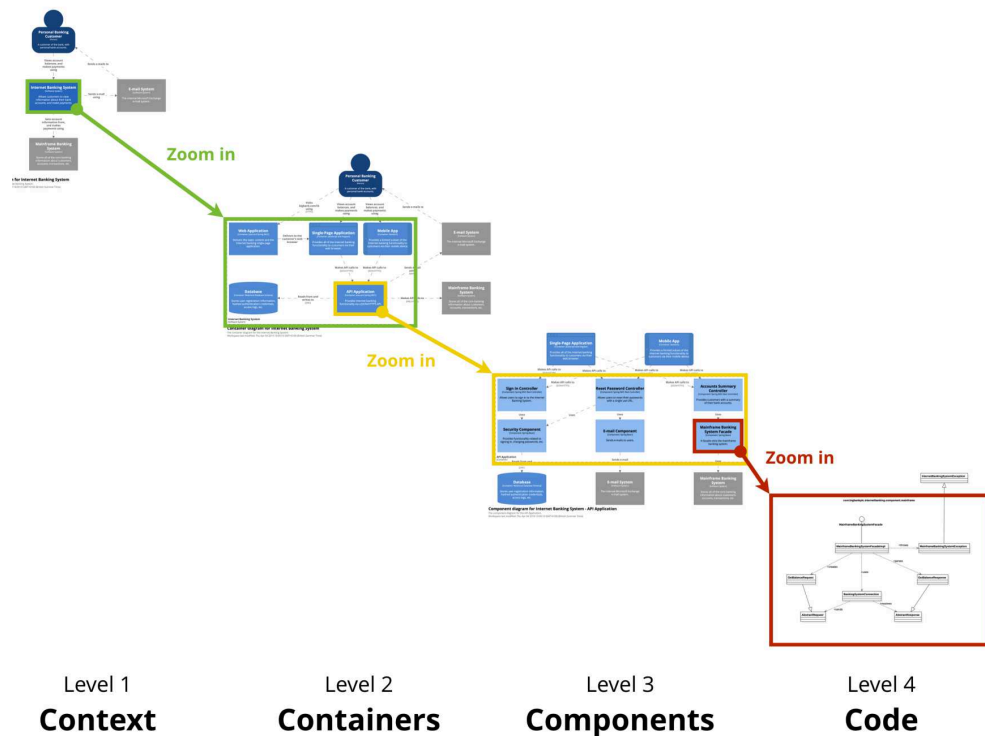


Zamknięcie RFC kończy się utworzeniem wynikowych ADR-ów — RFC jest procesem, ADR trwałym zapisem.

C4 model w skrócie

Simon Brown stworzył model C4 (Context, Container, Component, Code) do wizualizacji architektury — oparty na istniejących technikach modelowania (UML, diagramy relacji encji), a nie formalna notacja podobna do UML.

Poziom	Pytanie
System Context	Z kim i po co system współpracuje?
Container	Jakie deployowalne elementy mamy?
Component	Jak działa kluczowy kontener?



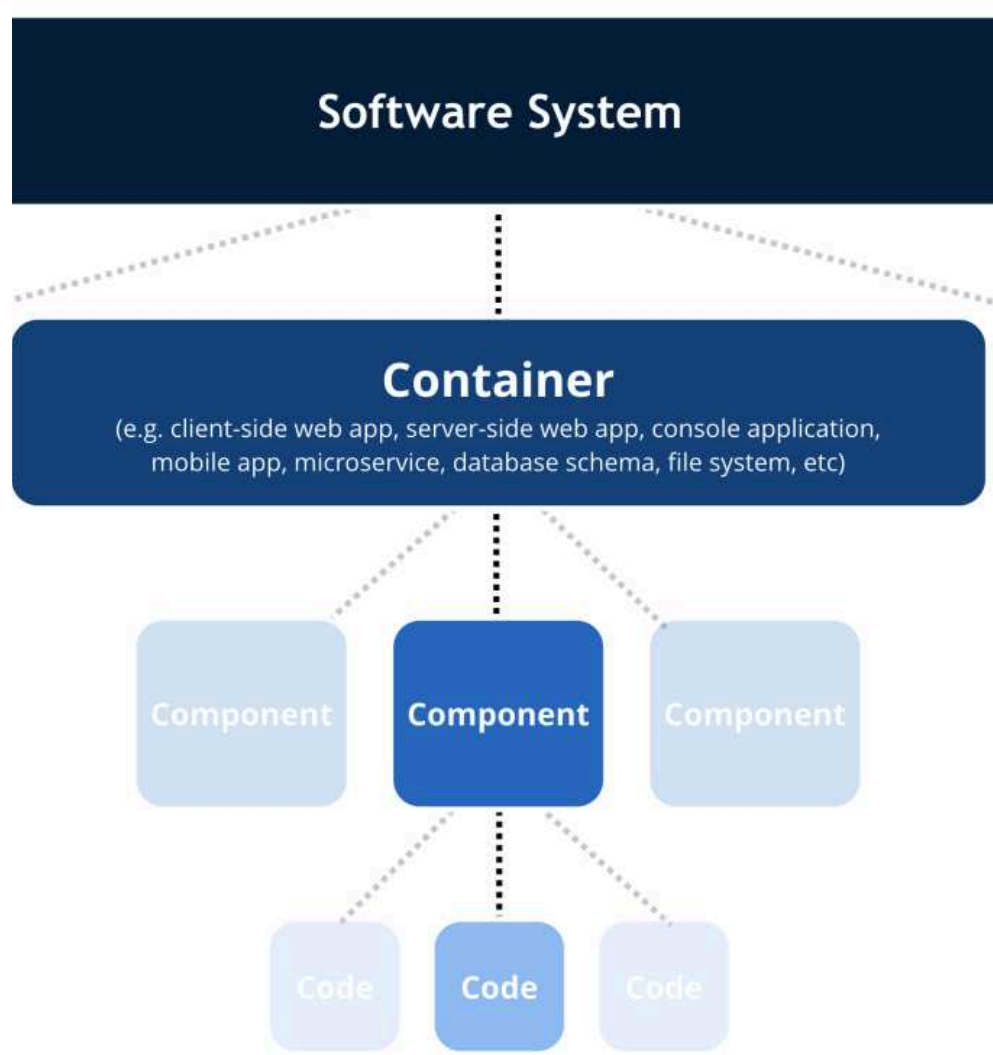
Poziom 1–2: System Context i Container

System Context

- Użytkownicy i role korzystające z systemu, systemy zewnętrzne i zależności
- Granica odpowiedzialności organizacji, opis celu każdej relacji — nie tylko strzałka

Container

- Aplikacje, usługi, procesy i magazyny danych, odpowiedzialność każdego kontenera
- Technologie istotne dla zrozumienia rozwiązania, komunikacja, protokoły i kierunek zależności



Poziom 3–4: Component i Code

Component — najważniejsze komponenty wewnątrz wybranego kontenera

- Wartościowy dla złożonego, kluczowego kontenera, który zespół faktycznie rozwija
- Zbędny dla prostych kontenerów albo gdy szybko wychodzi z synchronizacji z kodem

Code — klasy, interfejsy lub moduły implementacyjne

- Kod zmienia się szybciej niż diagram; diagram niezgodny z kodem jest gorszy niż brak diagramu
- Kod i testy są zwykle lepszą dokumentacją tego poziomu niż dodatkowy diagram

Diagramy uzupełniające i czytelność

- **Dynamic view** — przebieg wybranego scenariusza, **Deployment view** — rozmieszczenie w środowisku
- **System Landscape** — relacje między wieloma systemami organizacji
- Widok stanu obecnego i docelowego, granice bezpieczeństwa, sieci, zespołów i odpowiedzialności
- Nazwa i jednoznaczny typ każdego elementu, krótki opis odpowiedzialności
- Opis znaczenia relacji zamiast samej strzałki, konsekwentny poziom abstrakcji
- Legenda, czytelne granice, ograniczona liczba elementów na jednym diagramie

Utrzymanie modeli i typowe błędy C4

- Model jako kod, wersjonowany razem z dokumentacją; przegląd zmian diagramu w tym samym PR co zmiana architektury
- Powiązanie elementów C4 z repozytoriami, zespołami i usługami; sprawdzanie zgodności modelu z rzeczywistym systemem
- Jeden ogromny diagram próbujący pokazać wszystko; mieszanie systemów, kontenerów, komponentów i klas na jednym diagramie
- Nazwy będące wyłącznie technologiami, np. „Java” albo „Kafka”; strzałki bez opisu znaczenia
- Model docelowy prezentowany jako stan rzeczywisty

Praktyka warsztatowa

- Jedno RFC dla „pierwszy remote z monolitu” — 2–3 realne opcje (nie jedna „słuszna”)
- Wynik: ADR + plan migracji + metryki sukcesu
- Diagram C4 Container **przed i po** — pokazuje, co faktycznie się zmienia, nie tylko deklarację intencji
- ADR i RFC trzymane blisko kodu, w tym samym repozytorium co dokumentacja architektury

Modulith czy MFE?

Modulith czy MFE?

- to nie wojna religijna — to decyzja **kontekstowa**, zależna od zespołu, domeny i dojrzałości operacyjnej
- kolejność ma znaczenie: najpierw **granice domenowe**, dopiero potem **runtime split**
- „modulith first” zwykle obniża ryzyko: dobrze odizolowany moduł łatwo później wydzielić, źle podzielonych MFE nie da się łatwo połączyć z powrotem

Najważniejsza zasada

Mikrofrontendy rozwiązują przede wszystkim problem organizacyjnego skalowania rozwoju — nie są domyślnym sposobem na:

- dzielenie interfejsu na komponenty
- zmniejszanie rozmiaru bundle'a
- porządkowanie nieuporządkowanego kodu
- wprowadzanie wielu frameworków „na wszelki wypadek”

Narzędzie integracyjne ≠ dobra architektura

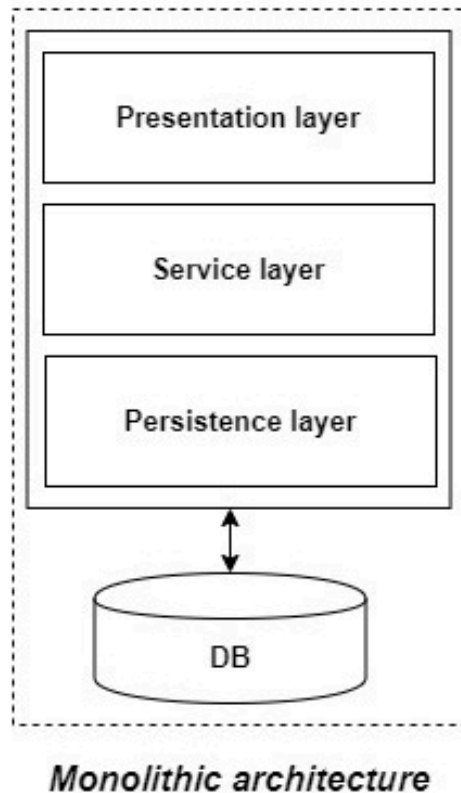


Modulith first

Modularny monolit często daje większość potrzebnych korzyści bez kosztu dystrybucji runtime:

- pionowe moduły biznesowe
- jawne API modułu i kontrolowane importy
- lokalny stan per moduł
- jeden prosty deployment
- łatwe refaktoryzacje granic, dopóki żyją w jednym repo/procesie

Decyzja **odwracalna**: dobrze odizolowany moduł można później wydzielić do osobnego MFE. Sklepanie z powrotem źle podzielonych MFE jest dużo droższe.



Kiedy modulith wygrywa

- 1–3 zespoły, wysoka zmienność domeny — granice mogą się jeszcze zmieniać
- potrzeba szybkich refaktorów przekrojowych (cross-cutting)
- brak gotowości operacyjnej na pełną dystrybucję (CI/CD, observability, rollback)
- chcemy zachować opcję późniejszego wydzielenia usług bez płacenia kosztu już teraz

Modulith — zalety

- prostsze wdrożenie i lokalne uruchamianie całości
- łatwiejsze refaktoryzacje wielu modułów naraz
- mniej problemów sieciowych i operacyjnych (jeden proces)
- granice gotowe pod późniejsze wydzielanie usług
- dobry domyślny wybór dla systemów back-office

Modulith — koszty i ryzyka

- wspólny cykl wydawniczy — release jednego modułu może czekać na inny
- możliwa stopniowa erozja granic bez świadomego egzekwowania (lint, CI)
- rosnący czas budowania i testowania wraz ze skalą
- trudniejsze niezależne skalowanie poszczególnych fragmentów
- wymaga dyscypliny zespołu — granice nie bronią się same

Kiedy MFE mają sens — dobre przesłanki

- kilka trwałych, autonomicznych zespołów
- granice biznesowe i wyraźny ownership end-to-end
- wspólny release realnie **blokuje** zmiany — nie tylko teoretycznie irytuje
- różne tempo rozwoju i różny poziom ryzyka poszczególnych obszarów
- stopniowa migracja dużego systemu (strangler)
- dojrzałe CI/CD i operacje (rollback, observability)

Kiedy MFE raczej nie mają sensu — słabe przesłanki

- moda lub chęć wypróbowania nowej technologii
- jeden mały zespół
- chęć reużycia kilku komponentów UI
- samo istnienie dużego repozytorium/kodu
- nieznana, szybko zmieniająca się domena
- brak realnej możliwości niezależnego wdrażania

Test sensownej granicy MFE

Usuń z opisu nazwy frameworka, bundlera i repozytorium. Co powinno zostać czytelne:

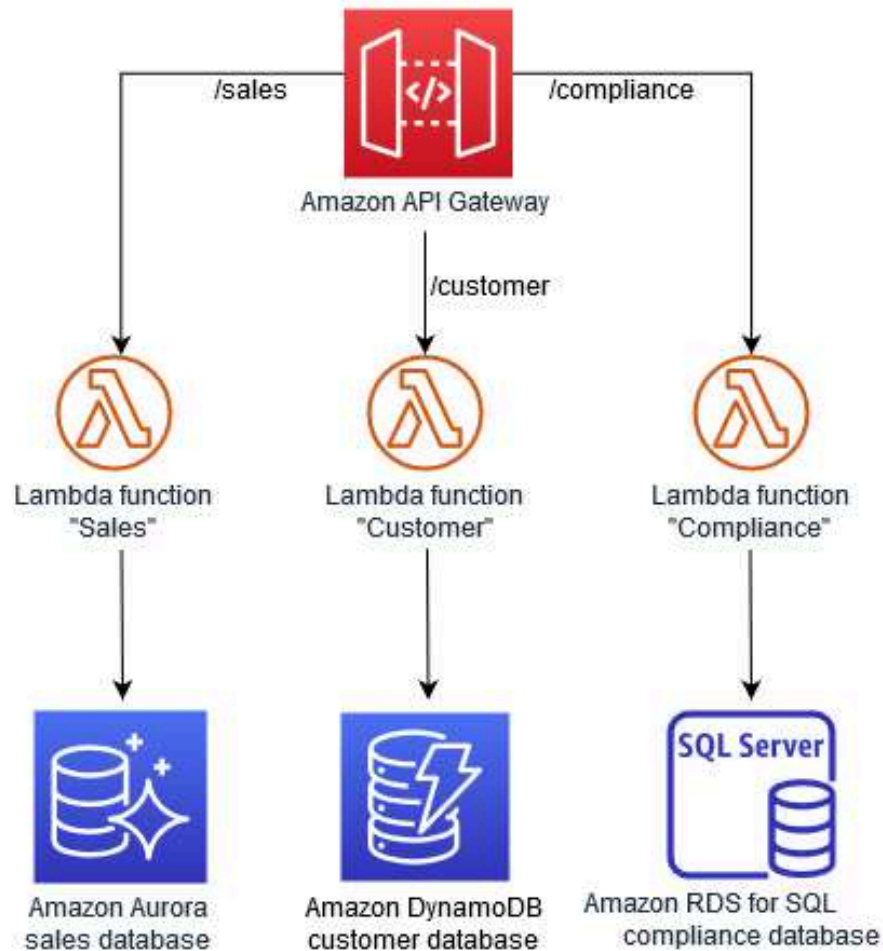
```
odpowiedzialność biznesowa → właściciel → publiczny kontrakt  
  → własność danych → niezależny cykl zmian  
  → sposób awarii i odtworzenia
```

Jeśli zostaje tylko „osobny projekt React” — to nie jest jeszcze granica architektoniczna.

Rozproszony monolit — pułapka złego wyboru

MFE bez realnej niezależności = najgorsze z obu światów: koszt operacyjny rozproszenia bez korzyści autonomii.

- skoordynowane wdrożenia wszystkich fragmentów mimo osobnych repo/buildów
- zmiana kontraktu wymusza aktualizację wszystkich konsumentów naraz
- wspólne biblioteki z modelami domenowymi wszystkich zespołów
- podział wg warstw technicznych (np. „header”, „widgets”) zamiast domen



Rozproszony monolit — skąd się bierze

- shell/frontend wykonuje orkiestrację wielu fragmentów zamiast delegować odpowiedzialność
- brak jasnej własności danych — każdy fragment czyta stan/API każdego innego
- przepływy użytkownika rozproszone po fragmentach bez zaprojektowanego procesu
- granice wycięte „siłą” pod istniejący kod, nie pod realną domenę

Analogia z backendu: mikroserwisy bez własnej bazy danych to rozproszony monolit z siecią zamiast wywołań lokalnych. W MFE odpowiednikiem jest fragment bez własnego, jasno przypisanego stanu i kontraktu.

Checklist: dobra granica czy rozproszony monolit?

- Czy mogę wdrożyć ten fragment bez koordynacji z innymi zespołami?
- Czy fragment ma własne dane/stan, których nikt inny nie czyta bezpośrednio z wnętrza?
- Czy zmiana kontraktu nie wymaga jednoczesnej aktualizacji wszystkich konsumentów?
- Czy awaria jednego fragmentu nie wyłącza całego ekranu?

Jeśli na którekolwiek pytanie odpowiedź brzmi „nie” — prawdopodobnie mamy rozproszony monolit, nie MFE.

Decision matrix

Kryterium	Modulith	MFE
Prostota operacyjna		
Autonomia wydań zespołów		
Koszt integracji i utrzymania		
Izolacja awarii UI		
Łatwość refaktoryzacji przekrojowej		
Niezależne skalowanie/tempo zespołów		
Odwracalność decyzji na start		

Rekomendacja procesu decyzyjnego

1. Zmapuj granice domenowe niezależnie od runtime.
2. Zapytaj: czy wspólny release realnie blokuje zespoły — dziś, nie hipotetycznie?
3. Sprawdź dojrzałość operacyjną (CI/CD, observability, rollback) — bez niej MFE = rozproszony monolit.
4. Zaczynij od Modulith, jeśli wątpliwości przeważają — to decyzja odwracalna.
5. Wydziel MFE tam, gdzie granica przechodzi test: właściciel + kontrakt + dane + niezależny cykl zmian.

Modularyzacja: granice i kontrakty

Modularyzacja: granice i kontrakty

- katalog to nie granica, kontrakt to granica
- niski coupling i wysoka kohezja jako cel projektowy
- enkapsulacja chroni decyzje, które mogą się zmienić
- publiczne API modułu musi być egzekwowane, nie tylko udokumentowane

Coupling jako zależność

- zależność kompilacyjna, runtime'owa, kontraktowa i organizacyjna
- coupling nie jest z definicji zły — problemem jest zależność niejawna, cykliczna lub trudna do zmiany
- liczy się nie tylko liczba zależności, ale ich siła i stabilność

Rodzaj	Objaw we frontendzie
Content	moduł sięga do wnętrza (<code>internal/*</code>) innego modułu



Symptomy i techniki ograniczania couplingu

Symptomy silnego sprzężenia

- import plików z wnętrza innego feature module
- jeden store ze stanem wszystkich domen
- event bus z nieudokumentowanymi nazwami i payloadami
- test jednostkowy wymagający uruchomienia połowy systemu

Jak ograniczać coupling

- publiczne API modułów + jawne reguły zależności
- fasady, porty i adaptery na granicach
- mapowanie modeli na granicach, jawne i wersjonowane kontrakty
- stan lokalny jako domyślny; consumer-driven contract tests

Enkapsulacja: co ukrywamy przed resztą

- cel: ukryć decyzje, które mogą się zmienić, i chronić niezmienniki modułu
- minimalne publiczne API + możliwość zmiany implementacji bez zmiany konsumentów
- komponent udostępnia props/zdarzenia, nie stan wewnętrzny; hook ukrywa mechanizm pobierania danych
- adapter ukrywający REST/GraphQL/SDK dostawcy; `package.json#exports` blokujący deep imports
- pułapka: barrel files są wygodne, ale mogą tworzyć cykle i nadmierne eksporty
- pułapka: wspólny store lub publiczny event bus mogą ominąć granice wszystkich modułów

Fasada ukrywająca store

```
// publiczne API modułu — nie eksportujemy store'a wprost
export function useCart() {
  const items = useCartStore(s => s.items)
  const addItem = useCartStore(s => s.addItem)

  return {
    items,
    total: items.reduce((sum, i) => sum + i.price, 0),
    addItem,
  }
}
```

- konsument zna wyłącznie `useCart()` — nie zna kształtu store'a ani biblioteki pod spodem
- implementację store'a można wymienić bez zmiany choćby jednego konsumenta
- publicznym API modułu jest `useCart`, nie wewnętrzny store

Tell, Don't Ask

```
// Ask – logika rozłana na zewnątrz modułu
if (order.status === 'pending' && order.canCancel) {
  order.status = 'cancelled'
}

// Tell – intencja i reguła w jednym miejscu
order.cancel()
```

- wywołujemy operację wyrażającą intencję zamiast pobierać dane i mutować je z zewnątrz
- `order.cancel()` / `cancelOrder(id)` zamiast ręcznej zmiany statusu w konsumencie
- reguły domenowe chronione wewnątrz właściwego modułu, nie rozproszone po UI
- nie każda struktura musi być „aktywnym” obiektem — czasem wystarczy czysta funkcja

Kontrakt modułu = publiczne API

```
// orders/index.ts – publiczne API modułu (barrel jako kontrakt)
export { OrdersPage } from './ui/OrdersPage'
export { useOrdersFacade } from './application/facade'
export type { OrderSummary } from './contracts'

// UWAGA: nic z ./internal ani ./infrastructure nie jest eksportowane
```

- konsumenci importują wyłącznie `@app/orders`, nigdy `@app/orders/internal/*`
- zmiana wewnętrznej struktury modułu nie wpływa na resztę aplikacji
- barrel file to **kontrakt**, nie przypadkowy eksport wszystkiego, co jest w katalogu
- kontrakt obejmuje UI, przypadki użycia (fasadę) i typy — nie store, nie klienta API
- katalog `internal/` (brak eksportu z indeksu) jako konwencja „to nie jest publiczne”

Anti-Corruption Layer

```
// mapper: DTO zewnętrznego API → model aplikacji
function toCustomer(dto: ErpCustomerDto): Customer {
  return {
    id: dto.cust_id,
    name: dto.full_name,
    createdAt: new Date(dto.created_ts),
  }
}
```

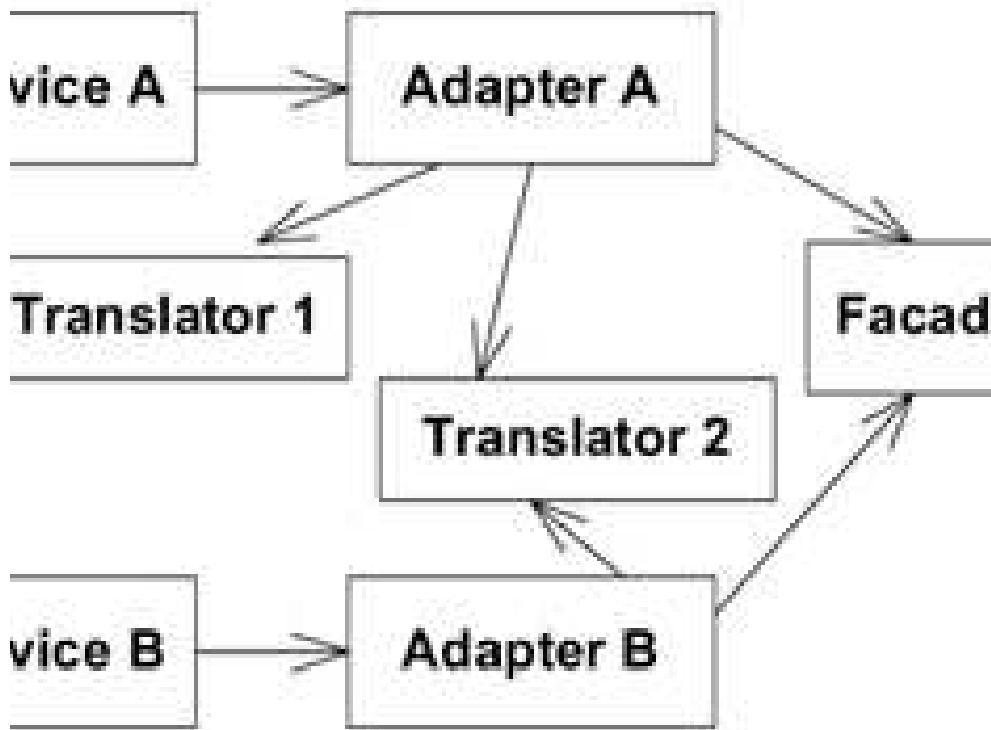
- warstwa tłumacząca między modelami, chroniąca integralność modułu przed modelem zewnętrznym/legacy
- tłumaczenie pojęć, struktur, identyfikatorów i błędów — nie tylko nazw pól
- walidacja odpowiedzi schematem runtime (np. Zod) — typy nie chronią danych z sieci
- lokalizacja: feature module (1 obszar), biblioteka integracyjna (wielu konsumentów) lub BFF — unikaj jednego globalnego ACL
- koszt: dodatkowe modele, kod mapujący i testy transformacji — to inwestycja, nie narzut

Anti-Corruption Layer

```
// mapper: DTO zewnętrznego API → model aplikacji
function toCustomer(dto: ErpCustomerDto): Customer {
  return {
    id: dto.cust_id,
    name: dto.full_name,
    createdAt: new Date(dto.created_ts),
  }
}
```

- warstwa tłumacząca między modelami, chroniąca integralność modułu przed modelem zewnętrznym/legacy
- tłumaczenie pojęć, struktur, identyfikatorów i błędów — nie tylko nazw pól
- walidacja odpowiedzi schematem runtime (np. Zod) — typy nie chronią danych z sieci

anticorruption layer



Boskie klasy (God Object)

- we frontendzie problem dotyczy komponentu, serwisu, store'a albo hooka, nie tylko klasy
- przykłady: komponent strony robiący wszystko naraz, globalny store z całym stanem aplikacji, `useEverything()` , shell znający szczegóły wszystkich remotes
- symptomy: duży plik, wiele niezależnych powodów do zmiany, test wymagający dziesiątek mocków, częste konflikty przy merge

Jak rozbijać

- podział według przypadków użycia i funkcji biznesowych, nie warstw technicznych
- oddzielenie orkiestracji (przypadek użycia) od prezentacji (komponent)
- stan lokalny lub stan feature'a zamiast jednego globalnego store'a
- refaktoryzacja etapami, zabezpieczona testami charakteryzującymi

Anemiczny model domeny

- obiekty domenowe zawierają wyłącznie dane, bez zachowania
- reguły biznesowe rozproszone po komponentach, serwisach i handlerach
- te same warunki implementowane wielokrotnie, niezależnie, na różnych ekranach
- niepoprawne stany mogą powstać bez żadnej kontroli

Jak to wygląda we frontendzie

- każdy komponent samodzielnie interpretuje statusy i uprawnienia
- każdy ekran inaczej oblicza dozwolone akcje dla tego samego obiektu
- UI pozwala wykonać operację, którą backend i tak odrzuci

Bogatszy model

```
type Order =  
  | { status: 'draft'; edit(): Order }  
  | { status: 'placed'; cancel(): Order }  
  | { status: 'shipped' }  
  | { status: 'cancelled' }  
  
function canCancel(order: Order): boolean {  
  return order.status === 'placed'  
}
```

- jawne typy opisujące dozwolone stany (discriminated unions) zamiast pola `string`
- funkcje domenowe / policy functions jako jedno źródło prawdy o regule
- Value Objects dla dat, pieniędzy i identyfikatorów

Ważne rozróżnienie

- bogaty model nie wymaga klas ani metod — funkcyjny model z czystymi funkcjami też działa
- DTO bez metod nie jest automatycznie „anemiczne”; prosty Transaction Script bywa właściwy dla CRUD
- frontend modeluje reguły potrzebne dla spójności UI, ale nie przejmuje roli źródła prawdy backendu

DIP i IoC: kierunek zależności i kierunek sterowania

```
interface OrdersRepository {  
  getById(id: string): Promise<Order>  
}  
  
// przypadek użycia zależy od portu, nie od fetch  
class CancelOrder {  
  constructor(private repo: OrdersRepository) {}  
  async execute(id: string) {  
    const order = await this.repo.getById(id)  
    return order.cancel()  
  }  
}
```

Pojęcie

Odpowiada na pytanie

DIP

jaki jest pożądany kierunek zależności?

DI

kto dostarcza konkretną implementację zależności?

IoC

kto steruje cyklem wykonania lub tworzenia?

Prawo Demeter i zasada Hollywood

```
// naruszenie – zna głęboką strukturę order  
const city = order.customer.address.city  
  
// zgodnie z zasadą – pyta o gotową wartość  
const city = getShippingCity(order)
```

- moduł zna wyłącznie swoich bezpośrednich współpracowników, nie ich wewnętrzną strukturę
- feature korzysta z fasady innego modułu, nie z jego repozytorium i stanu wprost
- **Hollywood** — „don't call us, we'll call you”: kod aplikacji rejestruje callback/plugin, a host wywołuje go we właściwym momencie (loader trasy, walidator formularza, lifecycle mikrofrontendu)
- granica zasady: odczyt z jawnej, nieziennej struktury nie zawsze jest problemem — celem jest ograniczenie wiedzy o cudzych szczegółach, nie eliminacja operatora kropki

Dobre granice w Retail Operations Portal

1. `Products` nie importuje wnętrza `Promotions` — tylko jego publiczne API
2. Integracja między modułami przez kontrakt i mapowanie modeli (ACL na granicy)
3. `shared` tylko dla stabilnych capability: auth, telemetry, design system
4. Store każdego modułu jest prywatny — na zewnątrz wychodzi fasada (`useOrdersFacade` , `useCart`)
5. Reguły cyklu życia zamówienia żyją w module `Orders` , nie w komponentach ekranów

Automatyczna ochrona granic

Automatyczna ochrona granic

- reguła bez egzekwowania to sugestia
- granice muszą zatrzymać build/CI
- potrzebujemy kilku poziomów ochrony

Granica istnieje dopiero wtedy, gdy jej naruszenie zatrzymuje build lub CI.

Katalog nie jest granicą

- `domains/orders/` wygląda jak moduł
- TypeScript nadal pozwala importować dowolny plik w środku
- `alias paths` tylko skraca ścieżkę — to odpowiedź dla kompilatora, nie izolacja
- `index.ts` bez kontroli pozostaje konwencją, nie kontraktem

Bez egzekwowania każda z tych konwencji jest tylko dobrą wolą zespołu.

4 poziomy ochrony

Poziom	Narzędzie	Co wykrywa
Plik	ESLint	deep imports, mieszanie runtime
Moduł	Nx / Sheriff	niedozwolone zależności między tagami
Graf	dependency-cruiser	cykle i naruszenia tranzytywne
Runtime	kontrakty, testy zgodności	niezgodne wersje API i eventów

Nie ma jednego narzędzia pokrywającego wszystkie poziomy — łączymy je w pipeline.

Dobór narzędzia w 30 sekund

Sytuacja	Zaczynij od
Nx monorepo	<code>@nx/enforce-module-boundaries</code>
Turborepo / pnpm	Sheriff lub <code>eslint-plugin-boundaries</code>
Jedna aplikacja	<code>eslint-plugin-boundaries</code>
Kilka prostych zakazów	<code>no-restricted-imports</code>
Kontrola pełnego grafu	<code>dependency-cruiser</code>

Reguły potrzebują języka architektury

Nie koduj polityki wyłącznie nazwami katalogów.

Wymiar	Przykładowe wartości
scope	orders, catalog, identity, platform
layer	domain, application, infra, ui
runtime	browser, server, neutral
deploy	shell, orders-mfe, shared

```
scope:orders      → scope:orders, scope:platform
layer:domain      → layer:domain, layer:shared
runtime:browser   !→ runtime:server
deploy:orders-mfe !→ deploy:catalog-mfe
```

Reguła scope i layer

```
scope: orders → orders, platform  
scope: catalog → catalog, platform
```

```
layer: ui → application, domain  
layer: application → domain  
layer: domain → neutralne utility
```

`orders` nie importuje implementacji `catalog` ; domena nie zna Reacta ani HTTP.

Reguła runtime i deploy

Wymiar	Może	Nie może
runtime:browser	neutralne kontrakty	kod Node, sekrety
runtime:server	logikę domenową	DOM, komponenty UI
deploy:remote	własną domenę, kontrakty platformy	źródeł innego remote, implementacji shell

Publiczne API jako granica

```
// źle: import wnętrza modułu  
import { OrderStore } from '@app/orders/internal/state';  
  
// dobrze: publiczny kontrakt  
import { OrderFacade } from '@app/orders';
```

- linter musi zabronić ominięcia publicznego entry pointu
- `package.json#exports` wzmacnia granicę — workspace package daje mocniejszą izolację niż zwykły folder

Nx: tagi i reguły zależności

```
{  
  "tags": ["scope:orders", "layer:application", "runtime:browser", "deploy:orders-mfe"]  
}
```

```
{  
  sourceTag: 'scope:orders',  
  onlyDependOnLibsWithTags: ['scope:orders', 'scope:platform']  
}
```

- ta sama reguła działa dla aplikacji, bibliotek i remotes
- `only ...` jest bezpieczniejsze niż długa lista zakazów

Sheriff: modularność bez Nx

Sheriff daje

Pasuje do

tagowanie modułów i macierz zależności

Vite, React, Angular, Vue

publiczne API, błędy zgłaszane w ESLint

Turborepo i pnpm workspaces

kontrolę z poziomu CLI

jednej dużej aplikacji, stopniowej modularyzacji

Nie mylić z **Sherif** (jedno „f”) — to linter spójności workspace (wersje zależności w `package.json`), nie narzędzie do modelowania granic domen.

ESLint: od zakazów do eslint-plugin-boundaries

- `no-restricted-imports` blokuje pojedyncze, znane wzorce importów — dobry start
- `eslint-plugin-boundaries` klasyfikuje pliki jako elementy architektury, porównuje źródło z celem importu

```
{  
  type: 'domain-module',  
  pattern: 'domains/*/.*',  
  capture: ['domain', 'layer']  
}
```

dependency-cruiser: cały graf zależności

- wykrywa cykle, moduły nieosiągalne oraz zależności tranzytywne
- zatrzymuje CI kodem wyjścia — to kontrola architektury, nie zamiennik feedbacku w IDE

```
{  
  name: 'no-cross-domain-internals',  
  severity: 'error',  
  from: { path: '^src/domains/([^/]+)/' },  
  to: { path: '^src/domains/(?!$1/)[^/]+/internal/' }  
}
```

Strategia wdrożenia w brownfield

Nie naprawiaj wszystkiego naraz:

1. wygeneruj aktualny graf zależności
2. nazwij domeny, warstwy i deployment units
3. zapisz reguły docelowe jako błędy
4. istniejące naruszenia zapisz jako baseline
5. blokuj każde nowe naruszenie od dziś
6. usuwaj baseline domena po domenie

Nowa architektura obowiązuje od dziś, nie „po refaktorze”.

Pipeline ochrony architektury

IDE / pre-commit	→ ESLint
Pull request	→ affected lint + test
CI	→ dependency-cruiser
Build	→ package exports
Integracja MFE	→ contract tests
Produkcja	→ telemetry kontraktów

Szybkie kontrole lokalnie, pełne kontrole w CI.

Najczęstsze antywzorce

- jedna biblioteka `shared` zależna od wszystkiego
- publiczne API eksportujące całe wnętrze modułu
- wildcardowe wyjątki „tymczasowe”
- reguły oparte wyłącznie na nazwach aplikacji, nie na tagach
- remote importujący store innego remote
- lint działający lokalnie, ale nie w CI

Najpierw zdefiniuj język granic (scope/layer/runtime/deploy). Dopiero potem wybierz narzędzie.

MFE: granice, ownership i shell

MFE: granice, ownership i shell

- MFE to model organizacyjny + techniczny — nie tylko sposób dzielenia kodu
- driverem jest autonomia zespołów, nie moda technologiczna
- granica musi mieć właściciela end-to-end, nie tylko właściciela repozytorium
- shell to platforma udostępniana zespołom, nie centralny „super-moduł”

Cel i zastosowanie

- Niezależny cykl budowania, testowania i wdrażania dla każdego zespołu.
- Rozwiązanie problemu **organizacyjnego** — wiele zespołów, jeden duży frontend.
- Podział według obszarów biznesowych (produkty, promocje, płatności), nie widgetów technicznych.
- Nie każdy SPA potrzebuje mikrofrontendów — to koszt, nie domyślna architektura.

Mikrofrontendy $\neq$ mikroserwisy

	Mikroserwisy	Mikrofrontendy
Granica	dane + logika biznesowa	UI + interakcja użytkownika
Driver	autonomia zespołu backendowego	autonomia zespołu frontendowego
Mapowanie	—	niekoniecznie 1:1 z usługami

Jeden mikrofrontend może korzystać z wielu mikroserwisów — i odwrotnie.

Sposoby kompozycji — przegląd

Build-time → Runtime (federacja modułów) → Server-side → Routing-level → iframe

- Każdy sposób to inny punkt na osi: siła izolacji ↔ łatwość integracji.
- Wybór zależy od tego, ile autonomii i izolacji naprawdę potrzebujemy — nie od mody (mechanika konkretnych technologii to osobny temat).

Build-time i runtime — dwa bieguny

- **Build-time:** mikrofrontend jako pakiet npm, budowany razem z hostem w jednym procesie CI — prosta konfiguracja, ale wspólny build i deployment to jeszcze nie autonomia wdrożeniowa.
- **Runtime:** mikrofrontend budowany i wdrażany **niezależnie**, host ładuje go dopiero w czasie działania aplikacji — prawdziwa niezależność release'ów kosztem złożoności integracji.

Server-side, routing-level, iframe

Tryb	Jak działa	Koszt / kiedy
Server-side	Fragmenty HTML składane przez serwer/edge (ESI, SSR streaming, server components)	Szybszy pierwszy render, trudniejsza interaktywność między fragmentami
Routing-level	Osobne aplikacje per trasa, reverse proxy/CDN kieruje wg ścieżki URL	Silna izolacja, ale pełne przeladowanie strony przy zmianie trasy
Iframe	Osobny dokument i kontekst JS, komunikacja tylko przez <code>postMessage</code>	Najsilniejsza izolacja, koszty UX/SEO/dostępności; dobre dla treści niezaufanych

Poziom izolacji a koszt integracji

Silniejsza izolacja

Łatwiejsza integracja

iframe

build-time integration

routing-level

runtime / federacja modułów

server-side (częściowo)

kompozycja po stronie serwera

Reguła: silniejsza izolacja zwykle oznacza trudniejszą komunikację i współdzielenie stanu.

Shell jako platforma — nie God Object

Shell decyduje, który mikrofrontend renderować i kiedy, udostępnia routing/layout/wspólne usługi — ale nie zna szczegółów implementacyjnych remote'ów, tylko ich kontrakt.

Powinien dostarczać	Nie powinien
composition i lifecycle remote'ów	trzymać logiki domenowej wszystkich remote'ów
routing top-level i layout	znać prywatnego stanu wewnętrznego każdego zespołu
auth context, telemetry, feature flags	pośredniczyć w każdej komunikacji między MFE
timeouty, fallback, zasady degradacji	wymuszać lockstep releases (wspólne wydania remote'ów)

Test: aktualizacja MFE bez zmiany kontraktu nie powinna wymagać zmian w shellu.

Kontrakty: shell ↔ remote

Co shell udostępnia

rejestrację routingu i punkty rozszerzeń

konfigurację oraz dane użytkownika

wspólne usługi przez jawne API, nie globalne zmienne

wersjonowanie kontraktu (remote deklaruje wspieraną wersję API shella)

Czego remote nie powinien robić

odwoływać się do wnętrza innego mikrofrontendu

zakładać, że jego wersja frameworka = wersja shella

ignorować obsługę własnej niedostępności (fallback jest w hoście)

manipulować globalnym DOM/CSS poza własną granicą

Sygnał na koniec: im więcej stanu globalnego i komunikacji między MFE, tym silniejsze sprzężenie granic — pełne wzorce (event bus, URL jako kontrakt, współdzielony store) to osobny temat warsztatu.

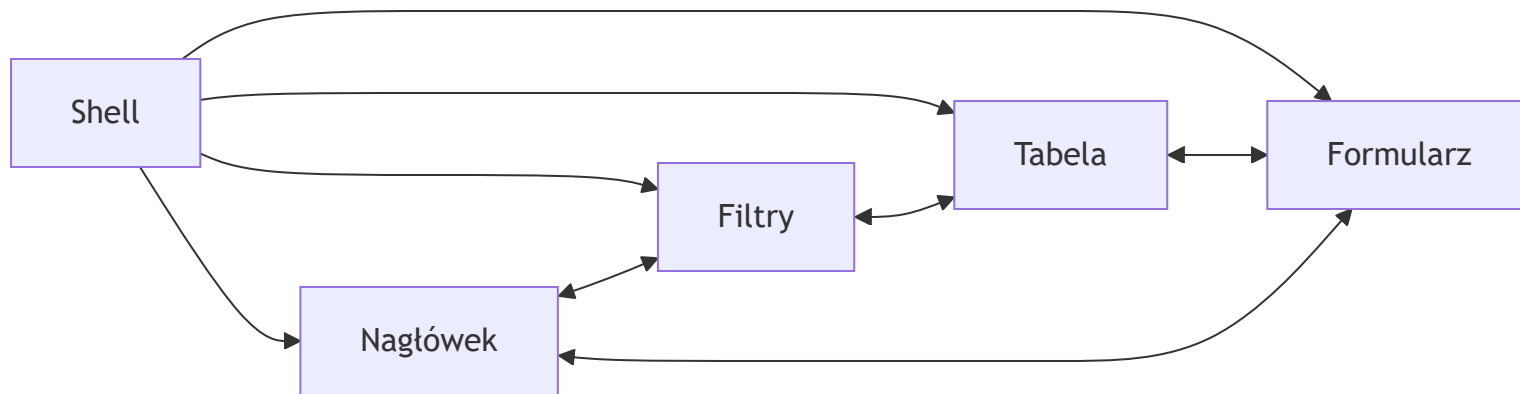
Wspólne elementy

- Design system, tokeny, spójność wizualna.
- Uwierzytelnienie i informacje o użytkowniku.
- Routing, nawigacja, obsługa błędów.
- Telemetria i korelacja zdarzeń.
- Lokalizacja i konfiguracja feature flags.
- Współdzielone zależności frameworkowe i konflikty wersji.

Ryzyka mikrofrontendów

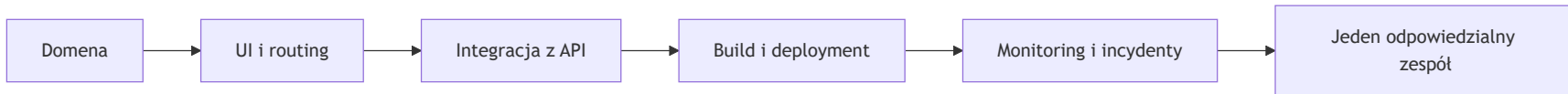
- Duplikacja bibliotek, wzrost rozmiaru aplikacji.
- Niespójny UX między granicami.
- Problemy z routingiem, SSR i hydracją.
- Globalne CSS i konflikty nazw.
- Trudne testy end-to-end.
- Niezależne zespoły nadal blokowane przez centralny shell.
- „Mikrofrontend z nazwy” — wdrażany wspólnie z całością.

Nano-frontendy → rozproszony monolit



Zbyt drobny podział granic (osobny MFE na nagłówek, filtry, tabelę) nie daje autonomii, gdy każdy fragment zna pozostałe, współdzieli stan i wymaga zsynchronizowanego release'u.

Ownership end-to-end



Ownership granicy to nie tylko kod — to również kontrakty i dane, pipeline i wersja produkcyjna, alarmy, rollback i dyżury operacyjne.

Kiedy mikrofrontendy mają sens

- Wiele niezależnych zespołów potrzebuje osobnego cyklu wdrożeniowego.
- Granice biznesowe są już dość stabilne, by je odizolować.
- Koszt operacyjny (duplikacja, złożoność) jest akceptowalny wobec zysku z autonomii.
- Driverem jest **organizacja**, nie chęć użycia najnowszej technologii — a jeden mały zespół zwykle ich nie potrzebuje.

Ownership canvas

Obszar	Owner	Co obejmuje
Products MFE	Team Catalog	UI, kontrakt, release, dyżur
Promotions MFE	Team Marketing Ops	UI, kontrakt, release, dyżur
Checkout MFE	Team Payments	UI, kontrakt, release, dyżur
Shell platform	Team Frontend Platform	composition, routing, auth, telemetry

Module Federation i Strangler Pattern

Module Federation i Strangler Pattern

- Module Federation — runtime-linkowanie niezależnie budowanych i wdrażanych aplikacji
- Strangler — ekstrakcja modułu z monolitu krok po kroku, bez „big bang”
- host (shell) ładuje remote w runtime, bez rebuildu przy każdej zmianie
- kontrakty (`exposes` , `shared`) i kompatybilność ważniejsze niż sam plugin

Punkt wyjścia: workspace

Zanim pojawi się federacja, moduły żyją w jednym repo jako workspace — linkowanie, nie kopiowanie.

```
// root package.json
{ "private": true,
  "workspaces": ["libs/*", "remotes/*", "shells/*"] }
```

- `npm install` w root linkuje każdy folder z `workspaces` do `node_modules/<name>`
- ten symlink rozwiązuje `import("@mfe/shared-store")` tak samo jak prawdziwą zależność z rejestru
- import relatywny (`../.. /libs/x`) wiąże kod z lokalizacją w systemie plików
- bare specifier (`@mfe/shared-store`) jest od tej lokalizacji niezależny — pakiet może później przenieść się do osobnego repo bez zmiany importów

Ścieżka ewolucji struktury

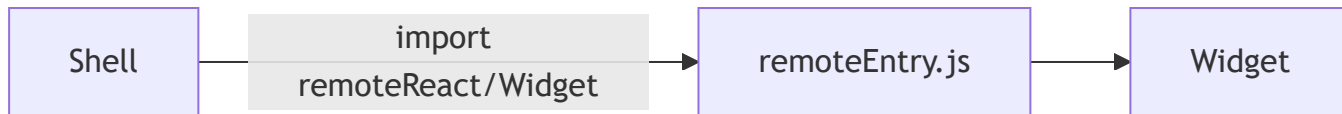
Path Aliases → Workspaces → Project References

Podejście	Opis	Kiedy stosować
Direct Export	eksport źródeł TS wprost w <code>package.json</code> , bez builda	moduły tylko wewnętrzne, nigdy niepublikowane
Prebuild	build modułu z wyprzedzeniem + wygenerowane typy	szybszy start, wymaga rebuildu po zmianie

- Project References: TypeScript przelicza tylko to, co się zmieniło — ok. 25s zamiast 175s przy pełnym cold buildzie dużego repo
- to naturalny punkt startowy przed przejściem na Module Federation — kod jest już podzielony na moduły z jawnym API

Słownik pojęć Module Federation

- Shell — host, łąduje inne aplikacje w runtime
- Remote — samodzielnie budowana i wdrażana aplikacja, `exposes` swoje moduły
- Container (`remoteEntry.js`) — wejście do remote'a, generowane przez plugin
- Expose — publiczny moduł remote'a, np. `./Widget`
- Shared scope — negocjacja wspólnych zależności (React, Redux...) w runtime



Shell nie zna kodu remote'a w czasie builda — tylko jego kontrakt: nazwę i listę `exposes` .

Ekstrakcja remote'a z modułu

Moduł, który wcześniej żył w monolicie, dostaje własny `webpack.config.js` i `exposes` :

```
new ModuleFederationPlugin({
  name: 'remoteReact',
  filename: 'remoteEntry.js',
  exposes: {
    './Widget': './src/Widget.tsx',
    './WebComponent': './src/webComponent.ts',
  },
  shared: { /* patrz dalej */ },
});
```

- ten sam kod, który wcześniej był importem względnym, staje się publicznym API remote'a
- `exposes` to dokładnie ten sam pomysł co barrel `index.ts` — jawna lista tego, co widać na zewnątrz
- `filename` to nazwa kontenera ładowanego przez shell jako `<script>`
- remote ma własny `webpack.config.js`, własny dev-server i własny deployment

Host statyczny: `shell-static`

```
const { container } = require('webpack');
const { ModuleFederationPlugin } = container;

new ModuleFederationPlugin({
  name: 'shellStatic',
  // URL każdego remote'a wpisany na stałe w build hosta
  remotes: {
    remoteJs: 'remoteJs@http://localhost:4001/remoteEntry.js',
    remoteReact: 'remoteReact@http://localhost:4002/remoteEntry.js',
    remoteVue: 'remoteVue@http://localhost:4003/remoteEntry.js',
  },
});
```

- to wbudowany w webpack5 `ModuleFederationPlugin` — zero dodatkowych zależności
- zmiana adresu remote'a wymaga rebuilda i redeployu shella
- `import('remoteReact/Widget')` w kodzie shella rozwiązuje się dzięki mapie `remotes` powyżej
- prosty start — ale wersja „na sztywno” nie skaluje się na wiele środowisk

Trzy strategie hosta

Shell	Jak deklarowany jest remote	Rebuild przy zmianie URL?
<code>shell-static</code>	<code>ModuleFederationPlugin.remotes</code> , czysty webpack5	tak
<code>shell-dynamic</code>	<code>registerRemotes()</code> w runtime, URL z <code>remotes.config.json</code> → <code>remoteEntry.js</code>	nie
<code>shell-manifest</code>	<code>registerRemotes()</code> w runtime, URL → <code>mf-manifest.json</code> (metadane remote'a)	nie

Remote budowany jest raz (`@module-federation/enhanced` , generuje też `mf-manifest.json`) — różni się tylko sposób ładowania po stronie hosta.

Wersja jako kontrakt

`shared` rozwiązuje przede wszystkim `coupling` wersji, nie `coupling` domen.

- bez `shared` : każdy remote wnosi własną kopię Reacta/Redux — większy bundle, możliwe konflikty
- ze `shared` : jedna kopia w runtime, negocjowana między hostem a remote'ami
- `singleton: true` wymusza **dokładnie jedną instancję** — kluczowe dla stanu (store, QueryClient)
- bez `singleton: true` na store'ze każdy remote dostałby swoją **własną kopię** stanu



mf-shared.js — jedno źródło prawdy

```
module.exports = {  
  react: { singleton: true, requiredVersion: false },  
  'react-dom': { singleton: true, requiredVersion: false },  
  '@reduxjs/toolkit': { singleton: true, requiredVersion: false },  
  '@tanstack/query-core': { singleton: true, requiredVersion: false },  
  '@mfe/shared-store': { singleton: true, requiredVersion: false },  
  '@mfe/shared-query': { singleton: true, requiredVersion: false },  
};
```

- współdzielony plik importowany przez każdy `webpack.config.js` (shell i remote) — ustawienia nie mogą się rozjechać
- `@mfe/shared-store` / `@mfe/shared-query` to własne paczki workspace'u — też mogą (i powinny) być singletonami
- `requiredVersion: false` upraszcza demo; produkcyjnie zwykle warto podać realny zakres (`^18.2.0`)

Dynamic: URL z runtime

```
import { init, registerRemotes, loadRemote }
  from '@module-federation/enhanced/runtime';

const urls = await fetch('/remotes.config.json')
  .then(r => r.json());

init({ name: 'shellDynamic', remotes: [] });
registerRemotes(
  Object.entries(urls).map(([name, entry]) => ({ name, entry })))
);

await loadRemote('remoteReact/WebComponent');
```

- zero URL-i w `webpack.config.js` — host jest niezależny od tego, gdzie remote aktualnie mieszka
- `entry` może pochodzić z API, feature-flagi albo configu per środowisko
- `loadRemote()` zastępuje magiczny `import('remoteX/ ...')` znany ze statycznej strategii
- manifest-shell różni się tylko tym, że `entry` wskazuje na `mf-manifest.json` zamiast `remoteEntry.js` — runtime sam rozpoznaje format

Lazy loading i routing

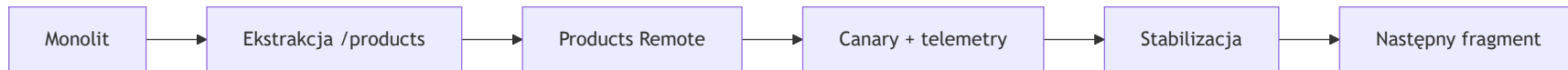
```
import { lazy, Suspense } from 'react';
import { Routes, Route } from 'react-router-dom';

const OrdersRemote = lazy(
  () => import('remoteOrders/OrdersPage')
);

<Routes>
  <Route
    path="/orders/*"
    element={
      <Suspense fallback={<Spinner />}>
        <OrdersRemote />
      </Suspense>
    }
  />
</Routes>
```

- `lazy()` + `import()` federacyjny to dokładnie ten sam mechanizm co code-splitting w monolicie — remote nie potrzebuje niczego specjalnego
- bundle remote'a pobiera się dopiero przy wejściu na trasę, nie przy starcie shella
- `Suspense` obsługuje czas ładowania sieci i czas ładowania `remoteEntry.js`

Strangler flow



Checklist ekstrakcji

- jawny `exposes` i publiczny kontrakt
- `shared / singleton` uzgodniony z resztą workspace'u (`mf-shared.js`)
- fallback, timeout, retry przy ładowaniu remote'a
- test kompatybilności shell ↔ remote
- remote buildowany i wdrażany niezależnie od shella
- rollback bez deploymentu całości

Web Components i heterogeniczność

Web Components i heterogeniczność

- Custom Element to kontrakt niezależny od frameworka hosta i remote'a.
- Dobrze dla mixed-framework i widgetów granicznych (zakupionych, legacy, zewnętrznych).
- Nie rozwiązują automatycznie problemów ownership — to wciąż osobna decyzja.

Custom Element jako neutralny kontrakt

- Rejestrujesz zwykły tag HTML (`customElements.define('price-widget', ...)`).
- Host nie musi wiedzieć, czy w środku jest React, Vue, Angular czy vanilla JS.
- Komunikacja wyłącznie przez atrybuty/właściwości (wejście) i zdarzenia DOM (wyjście).
- Shadow DOM izoluje style — ogranicza CSS leakage w obie strony.

Minimalny custom element

```
class PriceWidget extends HTMLElement {
  static observedAttributes = ['value', 'locale'];

  connectedCallback() {
    this.attachShadow({ mode: 'open' });
    this.render();
  }

  attributeChangedCallback() {
    this.render();
  }

  render() {
    const value = this.getAttribute('value');
    this.shadowRoot.innerHTML = `<span>${value}</span>`;
  }
}

customElements.define('price-widget', PriceWidget);
```

Zdarzenia jako kontrakt wyjściowy

```
this.dispatchEvent(new CustomEvent('submitted.v1', {  
  detail: { orderId },  
  bubbles: true,  
  composed: true, // przebija granicę Shadow DOM  
}));
```

- Nazwa zdarzenia wersjonowana (`submitted.v1`) — zmiana kształtu `detail` to nowa wersja.
- `composed: true` jest konieczne, żeby zdarzenie wyszło poza Shadow DOM do hosta.
- Host nasłuchuje `addEventListener` , niezależnie od frameworka, w którym zdarzenie powstało.

Kiedy to działa dobrze

- Migracja etapowa React/Vue/Angular — stary i nowy stos współistnieją pod jednym tagiem.
- Osadzanie małych, autonomicznych widgetów wymagających izolacji stylów (Shadow DOM).
- Zakupiony lub legacy system, którego nie chcesz/nie możesz przepisać.
- Silna granica domenowa albo odrębne wymagania techniczne zespołu.

Web Components dają neutralny kontrakt, ale **nie rozwiązują ownershipu**.

Web Component jako wspólny mianownik

- Shell może być czystym TS — nie zna Reacta ani Vue, tylko `customElements`.
- Remote eksportuje dwa wejścia: `./Widget` (surowy komponent frameworka) i `./WebComponent` (zarejestrowany custom element).
- Shell importuje wyłącznie `./WebComponent` i wstawia zwykły tag: `<remote-react-widget>`.
- Dzięki temu shell nie łąduje `react-dom` ani `vue` tylko po to, żeby coś wyrenderować.
- Remote, który *chce* być użyty jako natywne drzewo Reacta/Vue, wciąż ma do tego `./Widget`.

Bridge: dwa podejścia

Aspekt	well-known (np. <code>r2wc</code>)	hand-rolled
Ilość kodu	kilka linii, delegacja do biblioteki	pełna kontrola, więcej kodu własnego
Aktualizacja propsów	per-prop patchowanie	zwykle pełny re-render przy zmianie atrybutu
Zależności	dodatkowa biblioteka bridge'ująca	brak — tylko API frameworka (<code>createRoot</code>)
Kiedy wybrać	standardowy przypadek, mało czasu	potrzeba niestandardowego lifecycle

Bridge: przykłady kodu

```
// wellKnown.ts – deleguje do sprawdzonej biblioteki
import r2wc from '@r2wc/react-to-web-component';

export function defineReactWebComponent(tag, Component, props?) {
  customElements.define(tag, r2wc(Component, { props }));
}
```

```
// handRolled.ts – minimalna, ręczna wersja
class El extends HTMLElement {
  connectedCallback() {
    this.root = createRoot(this);
    this.root.render(createElement(Component, props));
  }
  disconnectedCallback() { this.root?.unmount(); }
}
```

`handRolled` upraszcza: brak per-prop patchowania, całe drzewo renderuje się od nowa przy każdej zmianie atrybutu.

Cena heterogeniczności

- Dodatkowe runtime'y i polyfills dla każdego frameworka na stronie.
- Większy transfer oraz zużycie pamięci przeglądarki.
- Więcej wariantów testów i SSR/hydration do utrzymania.
- Różne lifecycles i modele reaktywności do zsynchronizowania.
- Trudniejszy wspólny Design System między frameworkami.
- Wyższy koszt rotacji ludzi (każdy musi znać wszystkie stosy).
- Większe ryzyko niespójnego UX między widgetami.

Różnorodność jest kosztem do uzasadnienia, nie nagrodą za autonomię.

Kontrakt komponentu webowego

```
properties: value, locale, permissions  
events: submitted.v1, failed.v1  
styling: design tokens only
```

- `properties` to jedyne wejście — bez współdzielonego store'a czy kontekstu frameworka.
- `events` to jedyne wyjście — wersjonowane nazwy (`.v1`), `composed: true` , bubblujące do hosta.
- `styling` ograniczone do design tokenów — Shadow DOM chroni przed przeciekaniem CSS w obie strony.
- Ten kontrakt obowiązuje niezależnie od tego, czy w środku jest React, Vue czy Angular.

Design System cross-
framework

Design System cross-framework

- spójny UX wymaga wspólnego języka UI, niezależnie od frameworka mikrofrontendu
- współdzielimy tokeny i prymitywy, nie logikę domenową
- platform team usuwa tarcie, nie blokuje zmian

Warstwy design systemu

Warstwa	Współdzielenie	Przykład
Tokens	kolory, spacing, typografia	<code>color.brand.primary</code> , <code>space.4</code>
Primitives	button/input/dialog + a11y	focus ring, aria-*, kontrast
Patterns	formularze, tabele, feedback	walidacja, paginacja, toasty

Im niżej w tabeli, tym więcej kontekstu domenowego wycieka — patterns współdzielimy ostrożniej niż tokens.

Co warto współdzielić

- design tokens i kontrakty motywów (jasny/ciemny, marki)
- semantykę kolorów oraz typografii (nie surowe wartości hex)
- prymitywy dostępności — focus, aria, nawigacja klawiaturą
- wzorce interakcji (jak otwiera się dialog, jak zachowuje się błąd walidacji)
- ikony i testy wizualne (visual regression) jako część kontraktu
- dokumentację użycia — nie tylko Storybook, ale i „kiedy używać czego”

Kontrakt semantyczny komponentów

- komponenty DS eksponują intencję, nie implementację: `variant` , `intent` , `size`
- ten sam kontrakt semantyczny niezależnie od frameworka, w którym żyje MFE
- konsument nie powinien znać wewnętrznej struktury DOM/CSS komponentu
- zmiana wizualna (redesign) nie powinna wymagać zmiany kontraktu API

```
<Button variant="primary" intent="danger" size="sm" />
```

Wersjonowanie i okres zgodności

- design system wersjonujemy jak każdą współdzieloną zależność (semver)
- okres zgodności między wersją N i N-1, zanim wycofamy starą
- `latest` nie jest strategią wersjonowania — również dla tokenów i komponentów
- runtime update komponentu bez testu zgodności = ryzyko regresji w wielu MFE naraz

Governance: platform team

- usuwa tarcie — dostarcza gotowe, przetestowane prymitywy zamiast reguł do przestrzegania
- nie jest centralnym punktem blokującym każdą zmianę wizualną zespołów
- publikuje kontrakt i okres zgodności, nie „kolejkę pozwoleń” na merge
- odpowiada za a11y i testy wizualne prymitywów, nie za logikę biznesową patterns

Antywzorce

- domenowa logika (reguły biznesowe) wewnątrz komponentów DS
- globalny CSS bez warstw (`@layer`) i bez scope — kolizje specyficzności
- kilka resetów CSS i kopii tych samych fontów w różnych MFE
- centralny zespół blokujący każdą zmianę zamiast publikować kontrakt
- identyczny kod (copy-paste komponentu) mylony ze spójnym UX

Checklista

- ☐ tokens i prymitywy a11y są współdzielone, logika domenowa — nie
- ☐ każdy komponent DS ma jawny kontrakt semantyczny (variant / intent / size)
- ☐ wersje DS mają okres zgodności, latest nigdzie nie jest używane
- ☐ visual regression pokrywa krytyczne ścieżki UI
- ☐ platform team publikuje kontrakty, nie blokuje zmian

Komunikacja i shared state

Komunikacja i shared state

- domyślnie: stan lokalny — współdzielenie tylko przy udowodnionej potrzebie
- drabina decyzji: URL i kontrakty hosta zanim sięgniesz po event bus czy shared store
- każdy krok w prawo to mniej autonomii i więcej synchronizacji
- dziś: jak komunikować się między mikrofrontendami i kiedy naprawdę potrzebny jest współdzielony stan

Drabina decyzji komunikacyjnych

Wybierz najprostszy mechanizm, który rozwiązuje rzeczywisty problem.



Im dalej w prawo: więcej synchronizacji i wersjonowania, trudniejsze debugowanie, mniejsza autonomia zespołów. Nie przeskakuj szczebli "bo się przyda".

Kontrakty lokalne i kanały danych

Kontrakty lokalne

- URL — nawigacja, filtry, deep linki
- props / callbacki — jawny kontrakt shell ↔ MFE
- Custom Events, Web Components — rzadkie powiadomienia, properties + events
- event bus — tylko z katalogiem kontraktów, nigdy ad-hoc

Dane i kanały

- Backend API / GraphQL — trwałe stan biznesowy
- WebSocket / SSE — dane czasu rzeczywistego
- BroadcastChannel / Worker — synchronizacja między kartami
- localStorage — małe, niesekretne preferencje (nie baza domenowa)

Ważne rozróżnienia i asynchroniczność

Częste pomyłki

- komunikat **nie jest** stanem — event może zostać pominięty przez późnego konsumenta (nie ma historii)
- GraphQL nie jest magistralą UI ↔ UI, `localStorage` nie jest event busem
- ta sama biblioteka store'a w dwóch remote'ach **nie gwarantuje** jednej instancji w runtime

Asynchroniczność to kontrakt

Projektuj zachowanie dla wiadomości spóźnionej, utraconej, powtórzonej, dostarczonej w złej kolejności lub odebranej po unmount/reconnect odbiorcy — **happy path to za mało.**

Minimalny event bus

```
type Handler<T> = (payload: T) => void;

export function createEventBus() {
  const handlers = new Map<string, Set<Handler<any>>>();

  return {
    on<T>(event: string, fn: Handler<T>) {
      const set = handlers.get(event) ?? new Set();
      set.add(fn);
      handlers.set(event, set);
      return () => set.delete(fn); // sprzątanie na unmount
    },
    emit<T>(event: string, payload: T) {
      handlers.get(event)?.forEach((fn) => fn(payload));
    },
  };
}
```

- event to **fakt**, nie polecenie — nowy subskrybent nie zna wcześniejszej historii,
- zawsze zwracaj funkcję czyszczącą subskrypcję i wywołuj ją przy odmontowaniu,
- dobry do rzadkich powiadomień; do reaktywnego, współdzielonego stanu lepszy jest store.

Dobry kontrakt zdarzenia

```
interface PromotionApprovedV1 {  
  type: 'promotions.campaign-published.v1';  
  campaignId: string;  
  approvedBy: string;  
  occurredAt: string; // ISO 8601  
  correlationId: string;  
}
```

- nazwa domenowa i wersja (`promotions.campaign-published.v1`) — fakt biznesowy, nie polecenie dla odbiorcy
- mały, stabilny payload bez danych wrażliwych, jawny schemat i właściciel
- zmiany addytywne (breaking change = nowa wersja), correlation ID i znacznik czasu
- idempotentny odbiorca + obsługa duplikatów

Event spaghetti

- nieudokumentowane nazwy i payloady,
- łańcuch $A \rightarrow B \rightarrow C \rightarrow D$ bez jednego właściciela,
- wynik zależny od kolejności montowania komponentów,
- wielu autorów tego samego stanu,
- brak historii i aktualnego snapshotu dla nowego subskrybenta,
- subskrypcje pozostające po unmount (wycieki pamięci).

Eventy zmniejszają coupling tylko wtedy, gdy kontrakty są jawne i udokumentowane.

Najpierw sklasyfikuj stan

Rodzaj stanu	Preferowany właściciel / nośnik
Modal, walidacja, zaznaczenie	lokalny MFE
Nawigacja i udostępniany filtr	URL
Encje i reguły biznesowe	backend właściwej domeny
Cache danych serwerowych	lokalny cache MFE; współdzielony tylko z uzasadnieniem
Sesja i tożsamość	IdP + shell + wąski kontekst
Preferencje użytkownika	backend lub wersjonowana persystencja
Workflow cross-domain	jawny proces / orchestrator

Singleton store + QueryClient jako rdzeń

```
// libs/shared-store/store.ts
export const store = configureStore({
  reducer: { cart: cartReducer, user: userReducer },
});
export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;

// libs/shared-query/queryClient.ts
export const queryClient = new QueryClient({
  defaultOptions: { queries: { staleTime: 30_000 } },
});
```

- jedna instancja `store` / `queryClient` importowana wszędzie — singleton gwarantuje, że to naprawdę ten sam obiekt, nie kopia per remote
- `RootState` / `AppDispatch` to kontrakt TS między hostem a remote'ami; to zwykłe importy z paczki `workspace'u` — Vue korzysta z nich tak samo jak React

Providery per framework, ten sam singleton

```
// remote-react/Widget.tsx
<Provider store={store}>
  <QueryClientProvider client={queryClient}>
    <CartPanel />
  </QueryClientProvider>
</Provider>
```

```
// remote-vue/Widget.ts — Vue nie ma react-redux, subskrybuje store bezpośrednio
const observer = new QueryObserver(queryClient, userProfileQuery);
onMounted(() => {
  store.subscribe(() => { /* sync do ref() */ });
  observer.subscribe((r) => { /* sync do ref() */ });
});
onUnmounted(() => { /* sprzątanie subskrypcji */ });
```

Ten sam singleton, dwa sposoby "podłączenia się" — kontrakt (`RootState` , `queryClient`) jest wspólny, integracja z frameworkiem nie musi być.

Kiedy shared store ma sens

Warunki

- jeden kontrolowany runtime i shell
- częste aktualizacje małego, wspólnego modelu
- jeden właściciel zapisu, zarządzana zgodność wersji
- świadoma zgoda na mniejszą autonomię zespołów — kontrakt platformowy, nie stan całej domeny

Guardrails

- selektory i komendy zamiast surowego store'a, precyzyjne subskrypcje
- ograniczony zapis: jasno, kto i co może zmieniać
- wersjonowany kontrakt, odtwarzalny stan początkowy
- test bieżącej i poprzedniej wersji kontraktu + plan odejścia od store'a

Shared store — sygnały ostrzegawcze

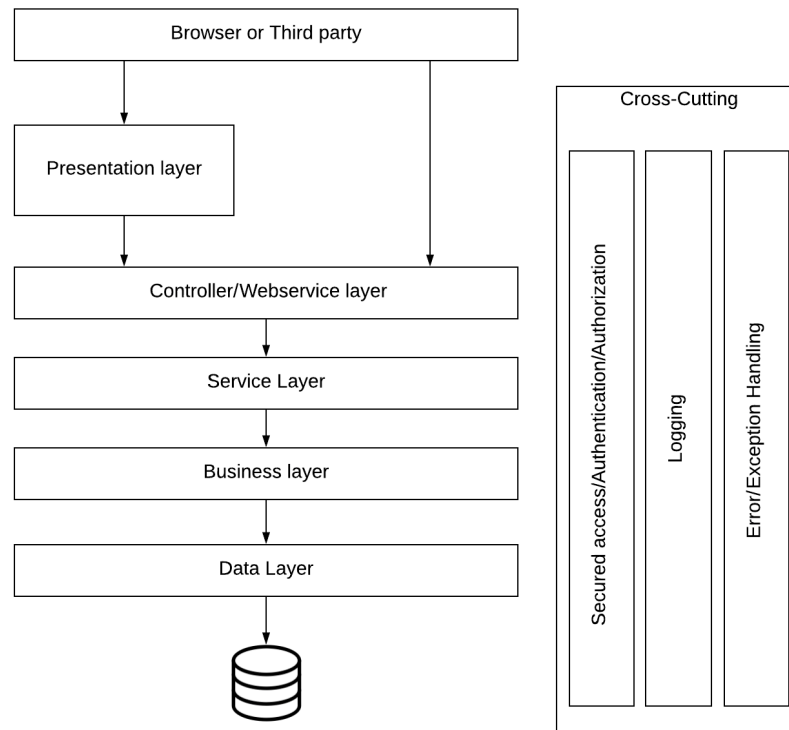
- każdy MFE zmienia dowolny fragment stanu,
- reducery zawierają reguły wielu domen,
- aktualizacja wymaga wspólnego release'u wszystkich remote'ów,
- store obsługuje także jednorazowe sygnały (to zadanie dla eventów),
- dane serwerowe kopiowane bez polityki ważności (staleness),
- usunięcie jednego MFE łamie niespokrewnione selektory w innych.

Globalny store łatwo omija wszystkie granice architektury, które próbowaliśmy postawić.



Spójność danych przy współdzielonym stanie

- strong vs eventual consistency — co naprawdę jest potrzebne? Spójność między cache klienta, shared store'em a backendem
- moment, w którym zapis staje się widoczny w odczycie u innego MFE; optimistic UI + rollback przy błędzie
- invalidation i rewalidacja cache po ewencie zmieniającym dane; read-your-writes dla własnych akcji użytkownika
- idempotencja operacji przy powtórzonych eventach/requestach; stan pending / confirmed / rejected widoczny w UI



Podsumowanie

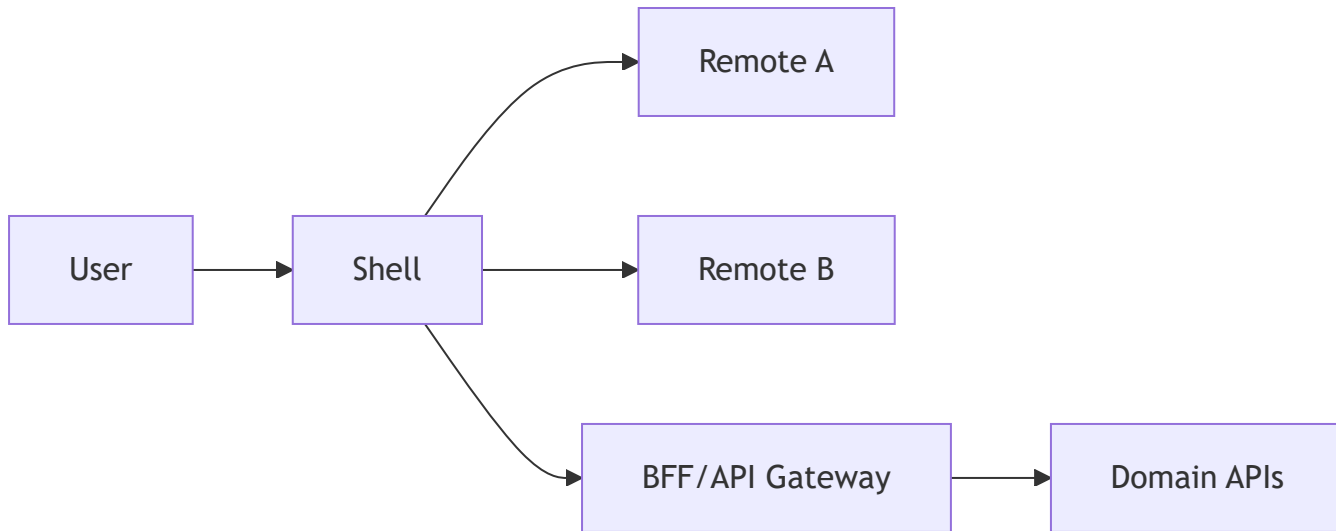
- zacznij od stanu lokalnego, idź w prawo drabiny dopiero z dowodem potrzeby,
- eventy: nazwa domenowa, wersja, mały payload, idempotentny odbiorca,
- shared store: jeden właściciel zapisu, guardrails, plan odejścia,
- projektuj dla wiadomości spóźnionej, utraconej i powtórzonej — happy path to za mało,
- świadomie wybieraj między strong a eventual consistency i komunikuj to w UI.

Auth i security boundaries

Auth i security boundaries

- granice bezpieczeństwa są równie ważne jak granice kodu
- authN/authZ egzekwowane po stronie backendu — nie tylko w UI
- shell przekazuje minimalny kontekst, nigdy sekrety
- remote działa z uprawnieniami strony hosta — kod „wewnętrzny” nie jest automatycznie zaufany

Model granic zaufania



Każda strzałka to osobna granica zaufania — wymaga jawnego kontraktu i polityki, nie tylko działającego kodu.

- Shell → Remote: jaki kontekst (rola, tenant, feature flags) faktycznie przechodzi dalej?
- Remote → BFF: token czy sesja? kto go odświeża i unieważnia?

Bezpieczeństwo runtime

Remote federowany w runtime działa z pełnymi uprawnieniami strony hosta — DOM, storage, sieć. To, że kod pochodzi z „naszego” repo, niczego nie gwarantuje.

- CSP i kontrolowana lista dozwolonych originów
- weryfikacja pochodzenia manifestów / `remoteEntry` przed załadowaniem
- minimalny kontekst (rola, id) zamiast surowych tokenów przekazywanych między MFE
- brak sekretów i tokenów sesyjnych w `localStorage`

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width,
    <title>Document</title>
  </head>
  <body>
    <div class="container">
      <div class="row">
        <div class="col">
          <h1>Welcome</h1>
          <p>
            Lorem ipsum dolor sit amet consectetur adip
            laborum, placeat ad labore impedit aspernat
            vero qui, rerum, accusamus eos incidunt, de
            asperiores!
          </p>
        </div>
      </div>
    </div>
    <script src="scripts/main.js"></script>
    <script src="scripts/plugins.js"></script>
  </body>
</html>
```

Kto odpowiada za co

Warstwa	Odpowiedzialność	Czego NIE robi
Shell	routing, przekazanie roli/tenantu	nie waliduje uprawnień biznesowych
Remote (UI)	ukrywa niedostępne akcje/widoki	nie jest źródłem prawdy o autoryzacji
BFF/API Gateway	authN, wymiana tokenów, rate limiting	nie ufa nagłówkom z UI bez weryfikacji
Domain API	authZ per operacja, audyt	nie zakłada, że UI już sprawdziło uprawnienia

UI może ukryć przycisk. Backend musi i tak odrzucić żądanie.

Security jako atrybut architektury

Bezpieczeństwo — ochrona danych, tożsamości i operacji — to jeden z atrybutów jakościowych, które trzeba świadomie zaprojektować w architekturze frontendu, a nie doklejać na końcu jako „hardening”.

W mikrofrontendach ten atrybut przecina granice modułów: każda nowa granica (remote, BFF, API) to nowa powierzchnia do zabezpieczenia.

Precyzyjny wymóg, nie hasło

Źle: System musi być bezpieczny.

Dobrze: Użytkownik bez roli `Finance` nie może odczytać danych płatności — ani przez UI, ani przez API, ani przez bezpośrednie zapytanie do warstwy raportowej.

Dopiero tak zapisany wymóg pozwala zaprojektować i przetestować konkretne elementy: authentication, authorization, network boundaries, data access, audyt, raportowanie.

Bezpieczeństwo ma cenę w użyteczności

Maksymalizacja samego bezpieczeństwa:

login → password → MFA → hardware key → approval

Formalnie bezpieczne. W praktyce użytkownicy zaczynają: współdzielić sesje, omijać proces, eksportować dane do Excela „żeby dało się pracować”.

Wniosek dla granic MFE: zbyt sztywna granica bywa obchodzona poza architekturą — i wtedy chroni mniej niż granica prostsza, ale realnie przestrzegana.

Case: `/approvals` w Retail Ops Portal

Wymóg (w stylu z poprzedniego slajdu): *Użytkownik bez roli `approver` nie może zatwierdzić ani odrzucić zamówienia — ani przez UI, ani przez wywołanie API `/approvals/:id/decision`.*

- UI (remote) — guard roli `approver` ukrywa przyciski approve/reject
- BFF/API — ponownie sprawdza rolę z tokenu, nie ufa samemu UI
- Audyt — każda decyzja (approve/reject) loguje: kto, kiedy, dla jakiego zamówienia
- Brak uprawnienia = brak akcji + wpis w logu bezpieczeństwa (próba nieautoryzowanego dostępu)

Checklist granic bezpieczeństwa dla MFE

- ☐ czy każda strzałka na diagramie granic ma jawny kontrakt autoryzacji?
- ☐ czy authZ jest sprawdzane na backendzie, niezależnie od UI?
- ☐ czy CSP i allowlist originów obejmują wszystkie remote'y?
- ☐ czy tokeny/sekrety nigdy nie trafiają do `localStorage` ani do telemetry?
- ☐ czy brak uprawnienia zostawia ślad audytowy, a nie tylko cichą odmowę?

Runtime remotes, wersje i
deployment

Runtime remotes, wersje i deployment

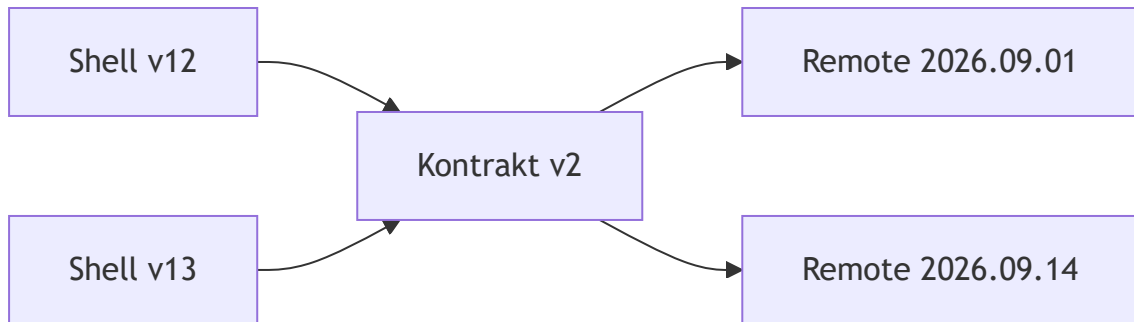
- niezależny deployment wymaga kompatybilności kontraktów, nie tylko wspólnego repo
- wersja artefaktu (build) $\neq$ wersja kontraktu (API, propsy, eventy, manifest)
- rollback musi być szybki, przewidywalny i nie wymagać koordynacji zespołów
- `latest` nie jest strategią wersjonowania

Model wersjonowania

Warstwa	Przykład	Zmienia się
Artefakt	<code>products-2026.09.01+sha</code>	przy każdym deployu
Kontrakt	<code>shell-api v2</code>	rzadko i świadomie
Shared deps	zgodny zakres semver	wg polityki singletonów
Remote entry	niezmienny URL / manifest	punkt wejścia jest stabilny

Artefakt i kontrakt żyją w innym tempie — to jest cel, nie problem.

Wersja wdrożenia $\neq$ wersja kontraktu



Bezpieczny runtime potrzebuje:

- niezmiennych adresów artefaktów
- przypinania wspieranej wersji kontraktu
- okresu zgodności i testów kontraktowych
- canary oraz szybkiego rollbacku

Runtime remotes — dobrze

- wystawiaj stabilne, publiczne entry points
- zachowaj asynchroniczny bootstrap — remote łąduje się niezależnie od shella
- remote traktuj jak zawodną usługę zewnętrzną
- ustal kompatybilne zakresy wersji dla shared deps
- singletony stosuj świadomie, nie domyślnie
- publikuj wersję i commit SHA przy każdym deployu

Runtime remotes — źle

- zawsze najnowszy remote (`latest`) bez możliwości rollbacku
- brak testu kombinacji shell-remote (stara wersja shella + nowy remote)
- setki wystawionych komponentów bez kontroli kontraktu
- lockstep release „na wszelki wypadek” zamiast realnej niezależności
- cykl zależności między remote’ami utrudniający pojedynczy rollback

Testy blisko kontraktu

1. jednostkowe — lokalna logika
2. kontraktowe — API, propsy, eventy, manifest
3. komponentowe — MFE + realistyczny harness
4. kompatybilności — bieżąca i poprzednia wersja kontraktu
5. E2E — kluczowe przepływy cross-domain
6. produkcyjne — smoke, synthetics, canary

Im bliżej kontraktu testujesz, tym rzadziej deploy zamienia się w koordynację zespołów.

Strategia rollout

Etap	Ruch	Decyzja
Canary	5–10%	metryki błędów i UX w normie?
Promocja częściowa	50%	brak regresji vs poprzednia wersja?
Pełny rollout	100%	monitoring nadal aktywny
Rollback	powrót do poprzedniej wersji	automatyczny przy naruszeniu SLO

Poprzednia wersja pozostaje „gorąca” aż do zakończenia pełnej promocji.

Niezależny release wymaga

- własnego pipeline'u CI/CD dla każdego MFE
- niezmiennych, atomowych artefaktów — bez nadpisywania wersji
- zmian kontraktu addytywnych lub okresu migracji równoległej
- feature flags ograniczających zasięg rollout
- canary mierzącego UX i błędy w czasie rzeczywistym
- automatycznego rollbacku bez ręcznej interwencji

Osobny pipeline bez niezależnego rollbacku to tylko pół autonomii.

Błędy, które zabijają autonomię

- `latest` zamiast pinowania konkretnej wersji
- brak testu „poprzednia vs nowa” wersja kontraktu
- brak fallbacku przy niedostępnym remote
- lockstep release „na wszelki wypadek”
- rollback wymagający koordynacji kilku zespołów naraz
- brak wersji i commit SHA w metadanych wdrożonego artefaktu

Observability i failure isolation

Observability i failure isolation

- observability to zdolność diagnozy, nie ilość zebranych danych
- telemetryka ma prowadzić do decyzji operacyjnej: rollback, hotfix czy dalsza obserwacja
- awaria jednego remote nie może zabić całego produktu
- mierzymy doświadczenie użytkownika, nie tylko infrastrukturę

Observability $\neq$ dużo danych

System jest obserwowalny, jeśli na podstawie sygnałów z zewnątrz potrafimy zrozumieć jego wewnętrzny stan.

- **Monitoring:** czy znany problem właśnie wystąpił?
- **Observability:** dlaczego system zachowuje się w ten sposób?

Telemetry to dane. Observability to zdolność diagnozy.

Pytania operacyjne po incydencie

- co nie działa i kogo to dotyczy?
- od kiedy i po jakim wdrożeniu?
- jaki jest wpływ i czy problem narasta?
- który komponent zawiódł, gdzie zniknął czas?
- kto może to naprawić — wycofać czy naprawiać?

Zaczynij od podróży użytkownika

Nie instrumentuj wszystkiego jednakowo.

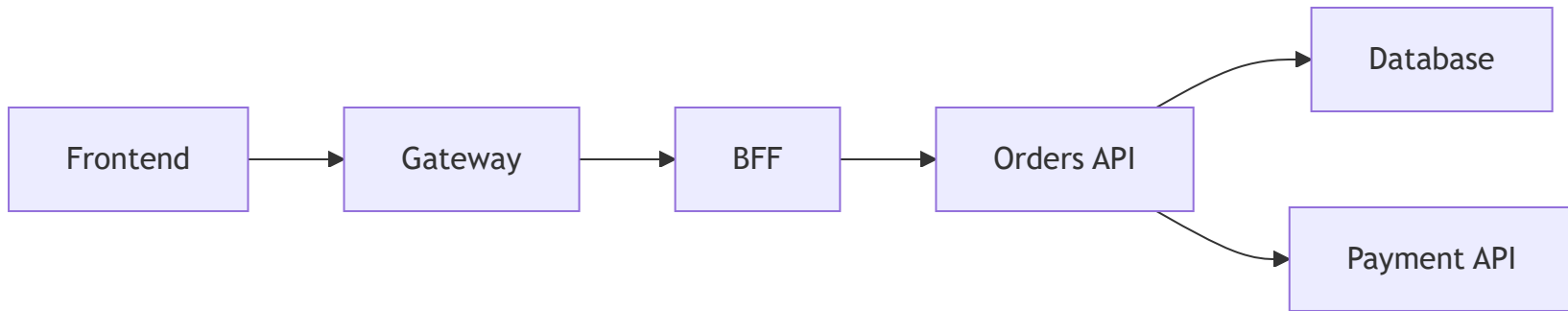
1. wybierz krytyczny przepływ
2. zdefiniuj poprawny rezultat
3. zmierz powodzenie i czas
4. połącz frontend z backendem
5. przećwicz awarię przepływu

Przykład: logowanie → wyszukanie → koszyk → zamówienie

Co warto obserwować?

Technicznie	Biznesowo
dostępność	udane logowania
latency p50 / p95 / p99	zapisane formularze
error rate	złożone zamówienia
zasoby i kolejki	wygenerowane raporty
zależności zewnętrzne	porzucone procesy

Mapa krytycznego przepływu



Na mapie oznacz właścicieli, SLO i punkty timeout/retry/degradacji.

Cztery sygnały razem

Sygnał	Odpowiada na pytanie
Metrics	jak duży jest problem?
Logs	co dokładnie się wydarzyło?
Traces	gdzie powstał problem?
RUM	co naprawdę zobaczył użytkownik?

Żaden pojedynczy sygnał nie daje pełnego obrazu — dopiero korelacja prowadzi do przyczyny.

Metryki usług i zasobów

RED (usługa / operacja)

USE (zasób)

Rate — liczba operacji

Utilization — wykorzystanie

Errors — udział błędów

Saturation — nasycenie

Duration — czas wykonania

Errors — błędy zasobu

Mierz RED per operacja biznesowa — CPU to wskazówka, nie miara doświadczenia użytkownika.

Średnia ukrywa problemy

Percentyl	Znaczenie
p50	typowe doświadczenie
p95	wolne 5% operacji
p99	ogon i trudne przypadki

- segmentuj po operacji, wersji, regionie i wyniku
- nie twórz wymiarów o nieograniczonej liczbie wartości, np. `userId` jako label

Logi jako zdarzenia

```
{  
  "level": "error",  
  "event": "order.submit.failed",  
  "service": "orders-api",  
  "version": "8f4a21d",  
  "traceId": "6d28 ...",  
  "errorType": "PaymentTimeout"  
}
```

Struktura umożliwia wyszukiwanie, korelację i agregację.

Log dobry i log słaby

Dobry

- stabilna nazwa zdarzenia, wynik i typ błędu
- service, version, traceId / spanId

Słaby

```
Something went wrong
```

Brak operacji, przyczyny i korelacji — sam tekst bez kontekstu.

Distributed tracing

Jeden trace pokazuje drogę operacji przez system:

```
POST /orders          1.8 s
├─ validate           8 ms
├─ database           120 ms
└─ payment-api        1.6 s ← problem
```

Trace wskazuje miejsce, log wyjaśnia przyczynę. Kontekst propagujemy przez W3C `traceparent` (np. `00-
<trace-id>-<span-id>-01`) — od interakcji frontendu, przez HTTP, po kolejki i joby.

RUM: perspektywa użytkownika

Frontend (RUM)

Backend nie wystarczy

JS errors i unhandled rejections

API może odpowiedzieć w 100 ms...

Core Web Vitals

...a użytkownik i tak czeka 5 s

nieudane requesty

na JS, mount remote'a i render

wynik ważnej akcji

zajęty main thread

Mikrofrontendy: kto co dodaje do telemetry

Shell ustanawia	Remote dodaje
sesję diagnostyczną	<code>remote.name</code> i <code>remote.version</code>
trace context	czas ładowania i mount
transport telemetryki	błędy <code>remoteEntry</code>
environment i release	błędy integracji z shellem

Korelacja i kontrakt telemetryczny MFE

Błąd użytkownika → trace → logi → release/commit → owner/runbook — przejście powinno wymagać kliknięcia, nie ręcznego kopiowania ID.

```
remote.name      remote.version
operation.name    operation.result
trace.id          error.type
deployment.environment  team.owner
```

Stabilne nazwy i wspólna semantyka dla wszystkich remotes — kontrakt wersjonowany jak API.

SLI, SLO i error budget

99,9% prób złożenia zamówienia zakończy się powodzeniem w czasie poniżej 2 s, w okresie 30 dni.

- SLI — zmierzony odsetek poprawnych operacji
- SLO — oczekiwany poziom, np. 99,9%
- Error budget — dopuszczalne niepowodzenia, tu 0,1%

Gdy error budget się wyczerpuje, priorytet przesuwa się z funkcji na stabilność.

Alertuj wpływ, nie każdy objaw

Słaby alert

Lepszy alert

CPU przekroczyło 80%

szybkie spalanie error budget

pojedynczy wyjątek

wzrost błędów krytycznej operacji

każdy status 404

trwała degradacja p95

brak właściciela i runbooka

regresja po wdrożeniu

Alert bez możliwej reakcji jest powiadomieniem albo szumem.

Izolacja awarii w mikrofrontendach

Awaria jednego MFE nie powinna oznaczać białego ekranu całego produktu.

- error boundary wokół każdego remote, loading/empty state i fallback UI
- timeout ładowania remote'a i danych, retry bez restartu produktu
- graceful degradation — reszta ekranu działa mimo braku jednej usługi
- circuit breaker ograniczający kaskadowe awarie, szybki rollback jako plan B

Scenariusz: gdy usługa rekomendacji nie odpowiada przez dwie sekundy, ekran produktu działa dalej bez rekomendacji, a błąd zostaje zarejestrowany.

Ćwiczenie: kontrolowana awaria

Stan: po wdrożeniu część klientów nie może złożyć zamówienia.

1. wyłącz `Products Remote`
2. sprawdź fallback shella — czy reszta produktu działa?
3. znajdź root-cause: dashboard SLO → trace → log → release marker
4. ustal wpływ, uszkodzony komponent i wersję, od której występuje problem
5. podejmij decyzję: rollback, hotfix, degradacja czy dalsza obserwacja?

Mierz MTTD i MTTR, ale oceniaj też jakość samej diagnozy.

Definition of Done: observability MFE

- ☐ krytyczne operacje mają nazwy i wyniki, logi są skorelowane przez `traceId`
- ☐ trace przechodzi przez granice shell → remote → backend
- ☐ dashboard pokazuje zachowanie użytkownika, SLO ma właściciela i error budget
- ☐ alert prowadzi do runbooka, release jest widoczny na wykresach jako marker
- ☐ awaria jednego remote nie blokuje reszty produktu — sprawdzone ćwiczeniem

MFE governance i migracja

MFE governance i migracja

- autonomia bez guardrails kończy się chaosem
- governance ma przyspieszać, nie blokować
- migracja to seria małych, mierzalnych kroków
- pytanie nie brzmi „ile MFE”, tylko „czy szybciej dowozimy wartość”

Po co w ogóle governance

- pełna autonomia zespołów = każdy wybiera własne standardy
- bez wspólnego mianownika UX i jakość produktu zaczynają się rozjeżdżać
- governance to **najmniejszy wspólny zestaw zasad**, nie centralna kontrola
- monorepo albo multirepo same w sobie nie dają ani autonomii, ani porządku

Minimum governance

- katalog MFE + jawny ownership każdego obszaru
- standard kontraktów (API, eventy) i ich wersjonowania
- quality gates: lint / testy / testy kontraktowe / compat
- polityka security i observability (szczegóły to osobny temat)
- ADR-y dla decyzji o granicach i komunikacji
- paved road: gotowe szablony, pipeline, telemetry „z pudełka”

Governance, które pomaga

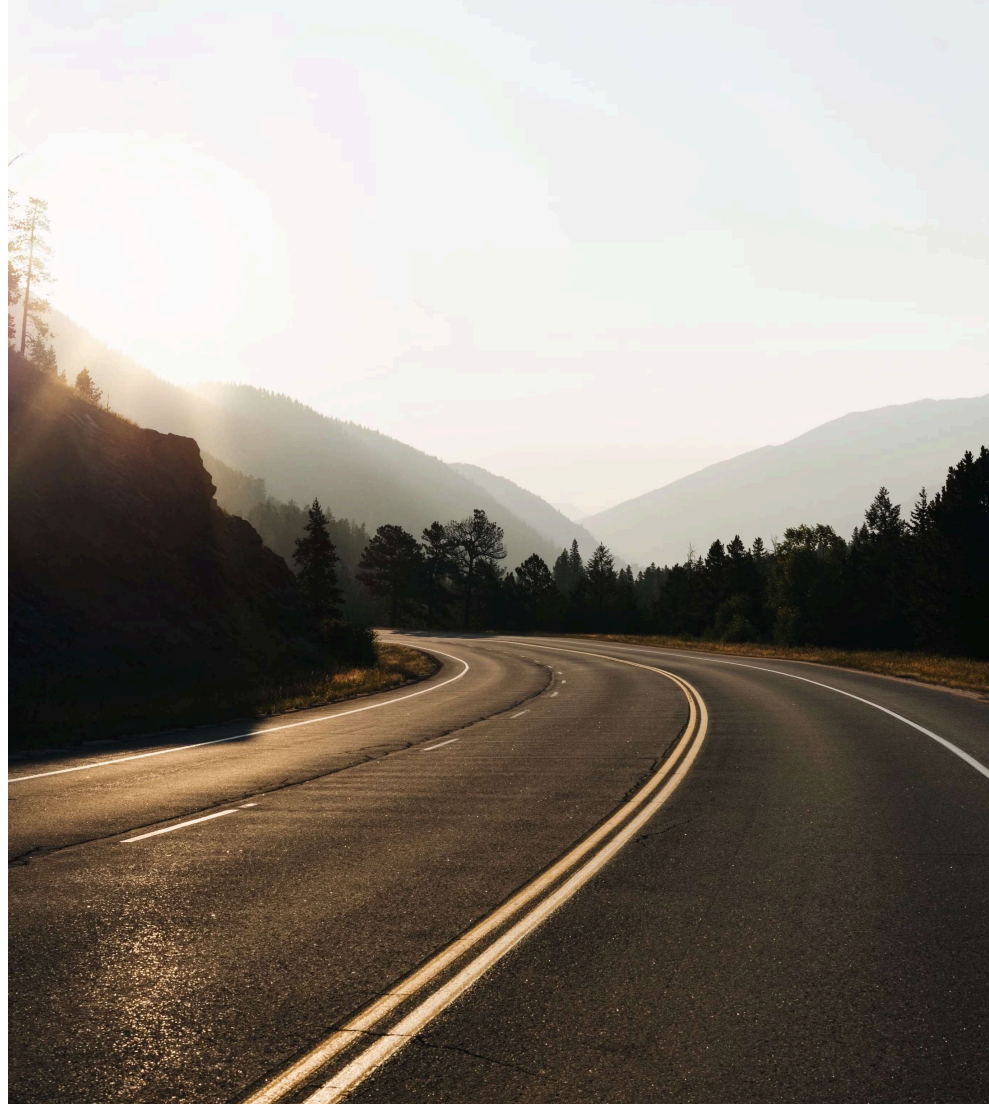
- mapa ownershipu i aktualny katalog MFE
- minimalne guardrails jakości i security, egzekwowane konsekwentnie
- cross-team review przy każdej zmianie kontraktu
- ADR-y dla granic i komunikacji między MFE
- paved road zamiast instrukcji rozproszonych po wiki
- regularne usuwanie starych eventów, flag i „tymczasowych” wyjątków

Governance, które blokuje

- pełna dowolność bez żadnych guardrails
- centralny owner wszystkich shared modules — wąskie gardło
- standardy istniejące tylko w dokumentacji, których nikt nie pilnuje
- sukces mierzony liczbą MFE zamiast efektu biznesowego
- lokalna autonomia zespołu kosztem spójności całego produktu
- brak ownera przepływów cross-domain
- każde odstępstwo od standardu musi przejść przez shell

Plan migracji brownfield

1. Zapisz drivery i miary sukcesu — po co to robimy i jak to zmierzmy.
2. Zmapuj domeny, zespoły, dane i zależności między nimi.
3. Najpierw uporządkuj granice **wewnątrz** monolitu.
4. Wybierz jeden pionowy fragment z jasnym ownerem.
5. Zbuduj cienki shell i jawny kontrakt integracji.
6. Wydziel MFE metodą Strangler Pattern — stopniowo, nie „wielkim wybuchem”.



Koegzystencja to część projektu

- przez tygodnie albo miesiące stary monolit i nowy MFE działają równolegle
- to nie stan przejściowy do zignorowania — to pełnoprawny etap projektu
- wymaga planu na: routing między starym i nowym, spójny UX, migrację danych
- pominięcie koegzystencji w planie = migracja urywa w połowie drogi
- brak rollbacku na tym etapie oznacza brak realnej autonomii wydania

Antywzorce — governance i granice

- MFE wprowadzone bez realnego problemu organizacyjnego do rozwiązania
- granice wyznaczone technologią lub warstwą, nie domeną biznesową
- shell jako God Object — wszystko musi przez niego przejść
- kontrakty i eventy bez właściciela i bez schematu
- decyzje o granicach podejmowane ad hoc, bez ADR

Antywzorce — organizacja i operacje

- każdy zespół inny stack/framework bez rachunku kosztów utrzymania
- Design System jako centralne wąskie gardło zamiast paved road
- sukces mierzony liczbą MFE albo rozmiarem bundle'a, nie efektem produktu
- osobne repozytorium mylone z realną autonomią zespołu
- brak cross-team review przy zmianie współdzielonego kontraktu

Przed dodaniem komunikacji między MFE

1. Czy granica MFE nadal jest właściwa?
2. Czy stan może pozostać lokalny dla jednego MFE?
3. Czy da się go odtworzyć z URL albo z API?
4. Czy wystarczy prosty kontrakt shell-MFE?
5. Czy potrzebujemy współdzielonego stanu, czy tylko pojedynczego sygnału?

Przed współdzieleniem stanu

6. Kto jest jedynym właścicielem zapisu?
7. Co zobaczy konsument, który dowie się o zmianie z opóźnieniem?
8. Co się stanie przy duplikacie, złej kolejności albo awarii?
9. Jak wersjonujemy i obserwujemy ten kontrakt w czasie?
10. Czy korzyść naprawdę uzasadnia utratę autonomii zespołu?

Shared store często ukrywa złą granicę, zamiast ją naprawiać.

Ryzyka i antywzorce — czerwone flagi

- shell staje się bottleneckiem dla wszystkich zespołów
- powstaje shared „super-lib” pełna logiki domenowej
- nikt nie jest ownerem przepływu cross-domain
- metryką sukcesu jest „liczba MFE”, a nie efekt biznesowy
- te same sygnały wracają na kolejnych retro — czas na ADR, nie kolejny wyjątek

Lekcje z wdrożeń

- najtrudniejsze problemy są społeczne: ownership i koordynacja, nie technologia
- autonomia potrzebuje małego, wspólnego zestawu guardrails
- wydajność, bezpieczeństwo i UX należą do całego produktu, nie pojedynczego MFE
- niezależny deployment ma wartość dopiero z detekcją i rollbackiem
- najlepszym punktem startowym bywa modularny monolit gotowy do wydzielenia

Podsumowanie

- governance to najmniejszy wspólny zestaw zasad, nie kontrola dla kontroli
- migracja brownfield to seria małych, mierzalnych kroków z koegzystencją w planie
- każdy antywzorzec z tego decku to sygnał: brakuje ownera albo brakuje ADR

MFE są dobre wtedy, gdy koszt integracji jest mniejszy niż koszt koordynacji, który usuwają

Podsumowanie i plan migracji

Podsumowanie i plan migracji

- architektura = świadome kompromisy, nie poszukiwanie jednego uniwersalnie „najlepszego” rozwiązania
- kolejność ma znaczenie: domeny → granice → egzekwowanie → runtime — odwrócenie tej kolejności to najczęstsza przyczyna nieudanych migracji
- sukces mierzymy zmianą w dwóch miejscach: przepływ dostarczania (lead time, częstość release'ów) i jakość (incydenty, dług architektoniczny)

90-dniowy plan wdrożenia (propozycja)

Okres	Cel	Sygnał sukcesu
0–30 dni	granice modułów + baseline naruszeń	granice zatwierdzone, CI raportuje naruszenia (na razie bez blokowania)
31–60 dni	pierwszy remote + fallback + kontrakty	1 obszar wdraża się niezależnie, kontrakt jest wersjonowany
61–90 dni	observability, rollout canary, governance	SLO monitorowane, pierwszy canary rollout bez incydentu

Faza 1: fundament (0–30 dni)

- domeny i właściciele zmapowani — jasno wiadomo, kto odpowiada za co
- granice modułów zapisane jako reguła egzekwowana narzędziem (lint/dep-cruiser), nie tylko w dokumencie
- CI raportuje naruszenia granic w trybie ostrzegawczym — budujemy baseline, jeszcze nie blokujemy
- wybrany pierwszy kandydat do wydzielenia: najmniejsze ryzyko, największa wartość

Faza 2: pierwszy remote (31–60 dni)

- pierwszy moduł wydzielony jako niezależnie budowany i wdrażany remote
- kontrakt (API / zdarzenia / komponenty) wersjonowany i publikowany osobno od implementacji
- fallback / graceful degradation na wypadek niedostępności remote'a
- CI: naruszenia granic przechodzą z trybu ostrzegawczego do blokującego dla nowego kodu

Faza 3: skala i governance (61–90 dni)

- observability z granulacją per moduł/zespół, nie tylko metryki globalne
- pierwszy rollout canary z automatycznym rollbackiem po przekroczeniu progu błędów
- governance: proces RFC/ADR dla kolejnych granic i kontraktów
- retrospektywa planu: co się potwierdziło, co wymaga korekty przed kolejnym kwartałem

Definition of Done migracji (wersja warsztatowa)

- co najmniej 1 obszar ma niezależny release — bez koordynacji z resztą monolitu
- zero nowych naruszeń granic w CI dla zmienianego kodu
- SLO zdefiniowane dla co najmniej 1 krytycznego flow + alerty podłączone i przetestowane
- playbook rollback przetestowany na realnym scenariuszu, nie tylko opisany na papierze
- dokumentacja granic i kontraktów aktualna i dostępna dla całego zespołu

Następne kroki po szkoleniu

1. wybór pierwszego strumienia migracji — właściciel i termin decyzji (np. 2 tygodnie)
2. RFC i ADR dla granic + kontraktów — recenzja z architektem i zespołem platformy
3. roadmapa techniczna i ownership rozpisane na 2 kolejne kwartały, zatwierdzone z biznesem

Najpierw działająca praktyka. Potem skala.